

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Universidad Abierta Interamericana
Facultad de Tecnología Informática

ShopMetrics

Informe de construcción del sistema

Semanas 1 a 6 del plan de trabajo

Alumno	Javier Gómez Avellaneda
Legajo	B00074234-T1
Carrera	Ingeniería en Sistemas Informáticos
Asignatura	Seminario de Trabajo Final
Comisión	5.º A — Turno Noche
Sede	Centro
Fecha	22 de septiembre de 2026

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática					
	Materia: Trabajo Final De Ingeniería					Fecha 22/09/2026
	Carrera: Ingeniería en Sistemas					
	Sede: Centro	Comisión: 5° A		Turno: Noche	Año: 2026	
ShopMetrics						

1. Sobre este informe

Este informe documenta la construcción del sistema ShopMetrics durante las primeras seis semanas del plan de trabajo presentado en la entrega anterior. No reemplaza a ese plan ni al documento de análisis y diseño: los toma como punto de partida y da cuenta de qué se construyó efectivamente contra cada tarea comprometida.

Cada semana ocupa un capítulo y cada tarea de esa semana, un apartado. Para cada tarea se deja constancia de seis cosas: el texto literal de la tarea en el plan de trabajo, lo que se hizo paso a paso, los archivos que quedaron escritos, las pruebas que los cubren, los commits que hicieron el trabajo y la tarjeta del tablero donde vive su seguimiento.

Los enlaces a los archivos no apuntan a la rama principal sino al commit exacto donde mirarlos. La diferencia importa: la rama principal sigue avanzando y en unos meses mostraría algo distinto de lo que este informe describe, mientras que un enlace fijado a un commit muestra siempre el mismo código.

1.1 Cómo se verifica lo que dice este informe

Todo lo que sigue es reproducible desde el repositorio. Con Docker levantado, estos cuatro comandos reconstruyen el entorno desde cero, corren las pruebas de los dos lados y verifican que el código, el esquema de la base y el documento de análisis digan lo mismo:

`make limpiar && make arriba` rehace la base desde las migraciones

`make probar` las pruebas de la API

`make probar-panel` las pruebas del panel

`make nivelar` cruza documento, esquema y código

1.2 Criterio para dar una tarea por terminada

Una tarea se cierra cuando cumple cuatro condiciones, no sólo la primera: el código está escrito y subido; hay pruebas que cubren todos los escenarios y no únicamente el que sale bien —cada curso alternativo de la especificación, cada validación por su propio motivo, los permisos y los límites—; la tarjeta del tablero tiene la evidencia con enlaces permanentes; y el control de nivelación contra el documento está en verde.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

2. Resumen de lo construido

Las seis semanas produjeron un backend, un panel web, dos simuladores de sistemas externos y un generador de datos, además de las herramientas de verificación que sostienen el resto.

Semana	Tema	Tareas	Archivos	Pruebas nuevas	Acumulado
1	Entorno de desarrollo y base de datos	3	14	0	0
2	Identidad y permisos	4	19	28	28
3	Ingesta de datos y generador de datos	3	14	39	67
4	Métricas del centro	3	9	63	130
5	Panel del administrador	3	18	53	183
6	Alertas por umbral	3	12	84	267

Tabla 1. Alcance construido por semana. Las pruebas se cuentan en la semana en que nació el archivo que las contiene. Fuente: elaboración propia.

La columna de archivos cuenta los que cada semana tocó, no sólo los que creó: un mismo archivo puede aparecer en varias semanas porque se lo siguió trabajando. Las pruebas, en cambio, se cuentan una sola vez, en la semana en que apareció el archivo que las contiene, y por eso sí suman.

2.1 Casos de uso construidos

De los 31 casos de uso especificados en el punto 10.5.3 del documento de análisis, diez tienen endpoints construidos y verificados. Cada endpoint cita en su documentación el caso de uso que materializa, de modo que la documentación que genera la API es también la trazabilidad contra el informe.

Caso de uso	Nombre	Semana
CU-001	Monitorear estado de las integraciones	4
CU-002	Registrar conector POS de un locatario	3
CU-003	Registrar dispositivo IoT de conteo	3
CU-004	Consultar mapa del centro y zonas	4
CU-006	Configurar alerta basada en umbral	6
CU-007	Recibir y gestionar alerta generada	6
CU-011	Consultar dashboard general y KPIs	4
CU-017	Autenticarse en la plataforma	2
CU-018	Configurar permisos granulares	2
CU-026	Monitorear centro de alertas críticas	6

Tabla 2. Casos de uso con endpoints construidos. Fuente: elaboración propia sobre el punto 10.5.3.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

3. Semana 1 · Entorno de desarrollo y base de datos

3.1 Lo que pedía el plan de trabajo

El plan de trabajo compromete para esta semana lo siguiente. La figura reproduce la página tal como está en el documento presentado, sin reescribirla.

Semana 1 · Entorno de desarrollo y base de datos

Sistema

- Repositorio, entorno de desarrollo y docker-compose con PostgreSQL
- Traducir el Diagrama Entidad-Relación (punto 10.5.8, 25 entidades) a las migraciones de la base.
- Definir las convenciones del código.

Negocio: Hoja de supuestos del modelo, FODA ampliado y definición de las tareas tercerizadas.

	Entregable
Sistema	Repositorio con el entorno levantando y la base de datos creada.
Negocio	Hoja de supuestos operativa y FODA ampliado.

Figura 3.1. Semana 1 del plan de trabajo. Fuente: planificacion_de_proyecto.pdf, página 5.

Entregable comprometido: Repositorio con el entorno levantando y la base de datos creada.

Punto de negocio de la semana: Hoja de supuestos del modelo, FODA ampliado y definición de las tareas tercerizadas.

El punto de negocio no forma parte de este informe, que cubre únicamente la construcción del sistema.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

3.2 S1.1 · Levantar el entorno de desarrollo con PostgreSQL y TimescaleDB

En el plan: «Repositorio, entorno de desarrollo y docker-compose con PostgreSQL.»

Tarjeta del tablero: [#1 — S1.1 · Levantar el entorno de desarrollo con PostgreSQL y TimescaleDB](#)

Qué se hizo

1. Se creó el repositorio con la estructura de carpetas: api/ para el backend, base/ para el esquema y la semilla, docs/ para la documentación técnica.
2. Se armó docker-compose.yml sobre la imagen timescale/timescaledb:latest-pg16, con volumen persistente y comprobación de salud.
3. Se escribió base/inicializar.sh. Hizo falta porque PostgreSQL sólo ejecuta los archivos sueltos en /docker-entrypoint-initdb.d y no entra en las subcarpetas: el script recorre migraciones/ y semilla/ en orden de nombre, así agregar una migración nueva es sólo dejar el archivo.
4. Se declararon las dos hipertablas de TimescaleDB —evento_trafico y registro_pos— en 002_series_temporales.sql. Son las dos tablas que crecen sin parar y TimescaleDB las parte por fecha.
5. Se comprobó que las hipertablas exigen su columna de partición dentro de la clave primaria, así que las dos quedaron con clave compuesta: (id, ts) y (id, fecha). No es una decisión de diseño nuestra sino un requisito del motor.
6. Se armó el Makefile con los comandos del día a día, para que levantar el entorno no dependa de recordar la línea de docker compose.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
docker-compose.yml	La base, adminer y —desde la semana 3— los simuladores	89c51d9
base/inicializar.sh	Recorre migraciones y semilla en orden de nombre	82ff853
base/migraciones/002_series_temporales.sql	Las dos hipertablas	89c51d9
Makefile	arriba, abajo, limpiar, base, logs, estado	89c51d9

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

.env.ejemplo	Las variables necesarias con valores de desarrollo	89c51d9
README.md	Cómo levantar el proyecto desde cero	89c51d9

Tabla 3.1. Archivos de la tarea S1.1. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
api/pruebas/test_postgres.py	11	Contra PostgreSQL real, en una base shopmetrics_prueba que se crea y se descarta en cada corrida. Verifican las 30 tablas, las 43 claves foráneas, que las dos tablas de serie sigan siendo hipertablas y la matriz de permisos de la semilla.

Tabla 3.2. Pruebas de la tarea S1.1. Fuente: elaboración propia.

Commits

[89c51d9](#) arranca el repositorio del sistema con el entorno y la base

[82ff853](#) corrige la creacion de la base al levantar el entorno

```

base/inicializar.sh 18 líneas
1  #!/bin/bash
2  # Postgres solo ejecuta los archivos que estan sueltos en /docker-entrypoint-initdb.d
3  # y no entra en las subcarpetas. Este script recorre las nuestras en orden, para que
4  # agregar una migracion nueva sea solo dejar el archivo en base/migraciones.
5  set -e
6
7  correr() {
8      local carpeta="$1"
9      [ -d "$carpeta" ] || return 0
10     for archivo in $(ls -1 "$carpeta"/*.sql 2>/dev/null | sort); do
11         echo ">> $archivo"
12         psql --username "$POSTGRES_USER" --dbname "$POSTGRES_DB" \
13             --set ON_ERROR_STOP=1 --file "$archivo"
14     done
15 }
16
17 correr /base/migraciones
18 correr /base/semilla
  
```

Figura 3.2. base/inicializar.sh. Fuente: elaboración propia.

Para tener en cuenta

- El runtime de contenedores en esta máquina es Colima, no Docker Desktop.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

- Verificado reconstruyendo la base desde cero (make limpiar && make arriba): las cuatro migraciones corren en orden y quedan 30 tablas, 2 hipertablas y 2 agregados continuos.

3.3 S1.2 · Traducir el diccionario de datos del punto 10.5.8 a las migraciones

En el plan: «Traducir el Diagrama Entidad-Relación (punto 10.5.8, 25 entidades) a las migraciones de la base.»

Tarjeta del tablero: [#2 — S1.2 · Traducir el diccionario de datos del punto 10.5.8 a las migraciones](#)

Qué se hizo

1. La primera versión del esquema se escribió a mano y quedó en 25 tablas.
2. Se reemplazó por un generador: base/generar_esquema.py lee la tabla del diccionario de datos del punto 10.5.8 del informe y produce el SQL. El esquema pasó de 25 a 30 tablas, 152 campos y 43 claves foráneas, que es lo que el diccionario realmente tiene.
3. Así el esquema no puede desviarse del documento sin que se note, y regenerarlo es un comando: make esquema.
4. Las cuatro restricciones UNIQUE van declaradas aparte en el generador, porque el diccionario no tiene forma de expresar unicidad. Cada una queda atribuida al caso de uso que la exige.
5. Se escribió api/nivelacion.py, que cruza diccionario, SQL, modelos ORM, índice de casos de uso y diagramas, y falla si no dicen lo mismo. Corre con make nivelar.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
base/generar_esquema.py	Lee el diccionario del Word y emite el SQL	89bdd35
base/migraciones/001_esquema_inicial.sql	30 tablas, 152 campos, 43 FK	89bdd35
api/nivelacion.py	Verifica que documento, esquema y código digan lo mismo	07b9e3c
docs/modelo-de-datos.md	Cómo se genera el esquema y por qué	89bdd35

Tabla 3.3. Archivos de la tarea S1.2. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
api/pruebas/test_postgres.py	11	Cuentan las 30 tablas y las 43 claves foráneas contra el esquema real, y verifican que el correo del usuario sea único en la base.

Tabla 3.4. Pruebas de la tarea S1.2. Fuente: elaboración propia.

Commits

[89bdd35](#) Generar el esquema desde el diccionario y completar las 30 entidades

[07b9e3c](#) Probar contra PostgreSQL y nivelar código con documento

base/generar_esquema.py	líneas 1 a 26 de 123
<pre> 1 """Genera el esquema de la base a partir del diccionario de datos del punto 2 10.5.8 del trabajo final, de modo que el modelo implementado y el documentado 3 no puedan divergir. 4 5 python3 base/generar_esquema.py <ruta del .docx> 6 """ 7 import sys, re, collections 8 from docx import Document 9 from docx.text.paragraph import Paragraph 10 from docx.table import Table 11 12 TIPO = {'UUID': 'UUID', 'VARCHAR': 'VARCHAR(160)', 'INT': 'INTEGER', 13 'TIMESTAMP': 'TIMESTAMPZ', 'DECIMAL': 'NUMERIC(14,2)', 14 'BOOLEAN': 'BOOLEAN', 'DATE': 'DATE'} 15 LARGOS = {'direccion', 'descripcion', 'mensaje', 'justificacion', 'checklist_resumen', 16 'accion_tomada', 'causa', 'condicion', 'valor', 'url_endpoint', 'api_key_enc', 17 'recurso', 'hash', 'rubro_objetivo'} 18 ALIAS = {'centro': 'centro_comercial', 'conector': 'conector_pos', 'dispositivo': 'dispositivo_iot', 19 'regla': 'regla_alerta', 'modelo': 'modelo_ml', 'prediccion': 'prediccion_vacancia', 20 'asignado_a': 'usuario'} 21 # TimescaleDB exige que la columna de particion forme parte de la clave 22 SERIES = {'evento_trafico': 'ts', 'registro_pos': 'fecha'} 23 24 # El diccionario de datos solo distingue PK y FK: no tiene forma de expresar 25 # unicidad. Estas restricciones salen de los casos de uso, que si la exigen, y 26 # se anotan aca con su origen para que se pueda rastrear. </pre>	

Figura 3.3. base/generar_esquema.py. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

```

api/nivelacion.py                                     líneas 1 a 26 de 142
1  """Control de nivelacion entre el codigo, el documento y los diagramas.
2
3  Cruza cuatro cosas que tienen que decir lo mismo:
4
5      el diccionario de datos (10.5.8) <=> el esquema SQL <=> los modelos ORM
6      el indice de casos de uso (10.5.2) <=> los endpoints que los materializan
7      los diagramas de clases por caso de uso (10.5.7) <=> los servicios del codigo
8
9  Se corre desde la raiz del repositorio: python3 api/nivelacion.py
10 """
11 from __future__ import annotations
12
13 import os
14 import re
15 import sys
16
17 RAIZ = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
18 DOC = os.path.join(RAIZ, '..', 'ShopMetrics-TFI')
19 WORD = os.path.join(DOC, 'documento', 'STF_Gomez_Javier_E1_v1.docx')
20 SQL = os.path.join(RAIZ, 'base', 'migraciones', '001_esquema_inicial.sql')
21
22 fallas: list = []
23
24
25 def control(etiqueta, valor, esperado=0):
26     estado = "OK" if valor == esperado else "REVISAR"

```

Figura 3.4. api/nivelacion.py. Fuente: elaboración propia.

Para tener en cuenta

- Las cuatro restricciones UNIQUE: usuario.email (CU-017-001), rol.nombre (CU-018-001 paso 2), permiso(recurso, operacion) (CU-018-001 paso 4) e indicador.nombre (CU-006-001).
- El plan dice 25 entidades y el diccionario tiene 30. Quedó anotado como DT-08.

Deuda técnica detectada

- [Tarjeta #51 del tablero](#)

3.4 S1.3 · Definir las convenciones del código

En el plan: «Definir las convenciones del código.»

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Tarjeta del tablero: [#3 — S1.3 · Definir las convenciones del código](#)

Qué se hizo

1. Se acordó el idioma: todo en castellano —tablas, campos, clases, funciones, variables y ramas—. Las únicas palabras en inglés son las que la herramienta impone. El motivo es práctico: el informe, el diccionario de datos y el código tienen que poder leerse juntos sin traducir nada en el medio.
2. Se fijaron las convenciones de base de datos: tablas en singular, clave primaria siempre id y UUID, foráneas como la tabla destino más _id, fechas en TIMESTAMPTZ, y migraciones numeradas que no se editan una vez aplicadas.
3. Se fijó la estructura de cada dominio —modelos.py, esquemas.py, servicios.py, rutas.py— y la regla de que las rutas validan y traducen a HTTP, y el servicio decide.
4. Se agregó el bloque de trazabilidad que lleva cada commit: a qué semana del plan responde, qué casos de uso materializa y qué puntos del informe toca.
5. Se escribió la definición de terminado: una tarjeta se cierra cuando el código está pusheado, hay pruebas de todos los escenarios, la tarjeta tiene evidencia con enlaces fijados a un commit, y make nivelar está en verde.
6. Se escribió herramientas/evidencia.py, que arma esos enlaces, corre las pruebas que le tocan a la tarea y publica el comentario en la tarjeta.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
CONVENCIONES.md	Idioma, base de datos, Python, commits y el DOD	fa068b7
herramientas/evidencia.py	Arma la evidencia y la publica en la tarjeta	5191942
herramientas/fichas.py	El inventario preciso de cada tarea	96c9d6e
herramientas/publicar_fichas.py	Publica la ficha y verifica los enlaces	96c9d6e

Tabla 3.5. Archivos de la tarea S1.3. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Commits

- [89c51d9](#) arranca el repositorio del sistema con el entorno y la base
- [5af3e86](#) Semana 2: identidad y permisos
- [fa068b7](#) Semana 3: generador de datos y simulador de sensores de conteo

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A		Turno: Noche	Año: 2026
ShopMetrics					Fecha 22/09/2026

[5191942](#) La herramienta de evidencia tambien corre las pruebas del panel

[96c9d6e](#) El inventario preciso de cada tarea, como dato

```

herramientas/evidencia.py                                líneas 1 a 26 de 126
1  """Publica la evidencia de una tarjeta del tablero.
2
3  Arma enlaces permanentes al codigo --fijados al commit, no a main, para que
4  sigan apuntando a lo mismo dentro de seis meses--, corre las pruebas que le
5  corresponden y deja el comentario en la tarjeta.
6
7      python3 herramientas/evidencia.py 8 \
8          --archivos api/app/dominios/integraciones/rutas.py \
9          --pruebas pruebas/test_integraciones.py \
10         --que "Alta de conector POS con validacion contra el sistema externo"
11  """
12  from __future__ import annotations
13
14  import argparse
15  import os
16  import re
17  import subprocess
18  import sys
19
20  REPO = "javieravellanedaa/shopmetrics"
21  RAIZ = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
22
23
24  def correr(args, cwd=RAIZ):
25      return subprocess.run(args, capture_output=True, text=True, cwd=cwd)

```

Figura 3.5. herramientas/evidencia.py. Fuente: elaboración propia.

Para tener en cuenta

- Los enlaces de la evidencia van fijados al SHA del commit y no a main, a propósito: así siguen apuntando a lo mismo cuando el trabajo se defiende.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

4. Semana 2 · Identidad y permisos

4.1 Lo que pedía el plan de trabajo

El plan de trabajo compromete para esta semana lo siguiente. La figura reproduce la página tal como está en el documento presentado, sin reescribirla.

Semana 2 · Identidad y permisos

Sistema

- Estructura del backend en FastAPI organizada por dominios: integraciones, alertas, privacidad y usuarios.
- Autenticación con JWT y los tres perfiles del sistema (CU-017).
- Control de acceso por rol sobre cada recurso, según la matriz de permisos (CU-018).

Negocio: Tamaño y valor del mercado con sus fuentes, y participación pretendida validada contra la capacidad de operación.

	Entregable
Sistema	API de autenticación y usuarios funcionando, con la colección de pruebas para ejecutarla.
Negocio	Cuadro de mercado y participación pretendida.

Figura 4.1. Semana 2 del plan de trabajo. Fuente: *planificacion_de_proyecto.pdf*, página 5.

Entregable comprometido: API de autenticación y usuarios funcionando, con la colección de pruebas para ejecutarla.

Punto de negocio de la semana: Tamaño y valor del mercado con sus fuentes, y participación pretendida validada contra la capacidad de operación.

El punto de negocio no forma parte de este informe, que cubre únicamente la construcción del sistema.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

4.2 S2.1 · Estructurar el backend en FastAPI por dominios

En el plan: «Estructura del backend en FastAPI organizada por dominios: integraciones, alertas, privacidad y usuarios.»

Tarjeta del tablero: [#4 — S2.1 · Estructurar el backend en FastAPI por dominios](#)

Qué se hizo

1. Se armaron los cuatro dominios que nombra el plan, cada uno con sus rutas, sus esquemas y su servicio.
2. Se separó la conexión a la base (base.py) de la configuración (configuracion.py), que se lee del entorno con Pydantic Settings.
3. Se estableció que cada endpoint cita en su resumen el caso de uso del punto 10.5.3 que materializa, así la documentación que genera FastAPI en /docs es también la trazabilidad contra el informe.
4. alertas e integraciones quedaron como armazón vacío, para que la estructura esté completa desde el principio y las semanas siguientes sólo agreguen contenido.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
api/app/main.py	Arma la aplicación y monta los cinco enrutadores	e660c3e
api/app/base.py	Motor, sesión y la clase madre de los modelos	5af3e86
api/app/configuracion.py	Todo lo que se lee del entorno	c099013
api/app/modelos.py	Los modelos ORM, que reflejan el diccionario	c099013
api/app/dominios/usuarios/rutas.py	Autenticación y usuarios	bc3c304
api/app/dominios/privacidad/rutas.py	Roles y permisos	bc3c304
api/requisitos.txt	Las dependencias, con versión fijada	69a388e

Tabla 4.1. Archivos de la tarea S2.1. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Commits

[5af3e86](#) Semana 2: identidad y permisos

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

```

api/app/main.py                                     líneas 1 a 26 de 60
1  """ShopMetrics - API.
2
3  El backend se organiza por dominios: usuarios y privacidad desde la semana 2,
4  integraciones desde la semana 3 y metricas desde la semana 4.
5  """
6  from contextlib import asynccontextmanager
7
8  from fastapi import FastAPI
9  from fastapi.middleware.cors import CORSMiddleware
10
11 from app import tareas
12 from app.configuracion import ajustes
13
14 from app.dominios.alertas.rutas import alertas
15 from app.dominios.integraciones.rutas import integraciones
16 from app.dominios.metricas.rutas import metricas
17 from app.dominios.privacidad.rutas import privacidad
18 from app.dominios.usuarios.rutas import autenticacion, usuarios
19
20 @asynccontextmanager
21 async def ciclo(_: FastAPI):
22     """Enciende la tarea periódica mientras el servicio está arriba."""
23     periodica = tareas.arrancar()
24     yield
25     if periodica is not None:
26         periodica.cancel()

```

Figura 4.2. api/app/main.py. Fuente: elaboración propia.

Para tener en cuenta

- Python 3.13.15. Se llegó a él después de que SQLAlchemy 2.0 fallara con Mapped[str | None] en Python 3.9: la anotación no se podía resolver.
- El stack: FastAPI, SQLAlchemy 2.0 con anotaciones Mapped[...], Pydantic v2 y pycpg 3.

4.3 S2.2 · Autenticación con JWT y los perfiles del sistema

En el plan: «Autenticación con JWT y los tres perfiles del sistema (CU-017).»

Tarjeta del tablero: [#5 — S2.2 · Autenticación con JWT y los perfiles del sistema](#)

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A		Turno: Noche	Año: 2026
ShopMetrics					Fecha 22/09/2026

Qué se hizo

1. Se implementó POST /autenticacion/ingresar, que devuelve el token, el usuario, sus roles, los centros a los que accede, sus permisos y a qué panel va.
2. El destino lo decide el backend y no el frontend, para que la regla del paso 5 del CU-017-001 viva en un solo lugar.
3. Las contraseñas se guardan con bcrypt directo, no con passlib: passlib 1.7.4 lee bcrypt.__about__, que ya no existe en bcrypt 4.1, y explota con AttributeError.
4. bcrypt trunca a 72 bytes, así que una contraseña más larga se pasa antes por SHA-256. Sin eso, dos contraseñas largas que compartan los primeros 72 bytes serían la misma.
5. Se implementaron las tres reglas de contraseña del paso 2 del caso de uso: ocho caracteres, una mayúscula y un número. Cada una falla por su propio motivo.
6. El mensaje de error no distingue entre correo inexistente y contraseña equivocada: decirlo regalaría la lista de correos registrados.
7. El acceso a los centros quedó como clase asociativa centro_acceso: un usuario accede a un centro *con un rol*, y esa terna es la fila.
8. Se escribió app/sembrar.py para crear el primer administrador sin tener que insertar a mano.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
api/app/seguridad.py	bcrypt, validación de clave y emisión del token	5af3e86
api/app/dominios/usuarios/servicio.py	Ingreso, roles y auditoría	5af3e86
api/app/dominios/usuarios/esquemas.py	El contrato del ingreso	5af3e86
api/app/sembrar.py	Crea el primer administrador	5af3e86

Tabla 4.2. Archivos de la tarea S2.2. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
--------------------	---------	------------

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

api/pruebas/test_autenticacion.py	7	El ingreso correcto, el correo que no existe, la contraseña equivocada, las tres reglas de la clave por separado y que el ingreso quede en auditoría.
---	---	---

Tabla 4.3. Pruebas de la tarea S2.2. Fuente: elaboración propia.

Commits

[5af3e86](#) Semana 2: identidad y permisos

api/app/seguridad.py	líneas 1 a 26 de 65
<pre> 1 """Claves y tokens de sesion. 2 3 Las reglas de la contraseña las fija el paso 2 del CU-017-001: minimo ocho 4 caracteres, con al menos una mayuscula y un numero. 5 """ 6 from __future__ import annotations 7 8 import re 9 from datetime import datetime, timedelta, timezone 10 from typing import Any 11 12 import bcrypt 13 from jose import JWTErrror, jwt 14 15 from app.configuracion import ajustes 16 17 # bcrypt corta a los 72 bytes. Las claves mas largas se resumen antes con 18 # sha256, asi ninguna pierde caracteres por el camino. 19 _TOPE = 72 20 21 22 def _preparar(clave: str) -> bytes: 23 cruda = clave.encode("utf-8") 24 if len(cruda) <= _TOPE: 25 return cruda 26 import hashlib </pre>	

Figura 4.3. api/app/seguridad.py. Fuente: elaboración propia.

Para tener en cuenta

- El plan dice «los tres perfiles» y el sistema tiene cuatro: Administrador del centro, Gerente, Operaciones y Locatario. Los cuatro salen del punto 10.5.3, que es la fuente principal, así que se implementaron los cuatro.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			Fecha 22/09/2026
	Sede: Centro	Comisión: 5° A	Turno: Noche	
ShopMetrics				

4.4 S2.3 · Control de acceso por rol sobre cada recurso

En el plan: «Control de acceso por rol sobre cada recurso, según la matriz de permisos (CU-018).»

Tarjeta del tablero: [#6 — S2.3 · Control de acceso por rol sobre cada recurso](#)

Qué se hizo

1. Se implementó exige(recurso, operacion), que devuelve una dependencia de FastAPI y corta el pedido con 403 si falta el permiso. El permiso queda a la vista en la firma del endpoint, no escondido en el cuerpo.
2. Primero inventé la matriz: siete recursos y dos operaciones. Al revisar el paso 4 del CU-018-001 resultó que la especificación enumera exactamente cinco recursos y cuatro operaciones. Se adoptaron los del documento.
3. La matriz quedó en veinte celdas, una fila de permiso por celda, con UNIQUE (recurso, operacion) para que no pueda haber dos.
4. Se cargó la matriz por rol en la semilla, con criterio de mínimo privilegio.
5. Se corrigió docs/api.md, que había quedado documentando los recursos viejos.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
api/app/dependencias.py	usuario_actual, permisos_de y exige	e333212
api/app/dominios/privacidad/rutas.py	Los cinco recursos y las cuatro operaciones	bc3c304
base/semilla/001_datos_base.sql	La matriz por rol	c099013
docs/api.md	La matriz documentada	36af13c

Tabla 4.4. Archivos de la tarea S2.3. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
api/pruebas/test_permisos.py	10	Verifican los dos lados: que el rol que debe pueda, y que el rol que no debe no pueda. Probar sólo el camino feliz dejaría pasar un control que nunca niega nada.

Tabla 4.5. Pruebas de la tarea S2.3. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

Commits

[5af3e86](#) Semana 2: identidad y permisos

[bc3c304](#) Alinear recursos y operaciones con el paso 4 del CU-018-001

[36af13c](#) Corregir la matriz de permisos en la documentación de la API

```

api/app/dependencias.py                                     líneas 1 a 26 de 69
1  """Piezas que las rutas piden por inyeccion: la sesion, el usuario que entro
2  y el permiso que hace falta para cada cosa.
3
4  El control por recurso es lo que pide el CU-018-001: cada endpoint declara que
5  permiso exige y el sistema lo resuelve contra los roles del usuario.
6  """
7  from __future__ import annotations
8
9  import uuid
10 from typing import Optional
11
12 from fastapi import Depends, HTTPException, status
13 from fastapi.security import OAuth2PasswordBearer
14 from sqlalchemy.orm import Session
15
16 from app.base import dar_sesion
17 from app.modelos import Usuario
18 from app.seguridad import leer_token
19
20 esquema_token = OAuth2PasswordBearer(tokenUrl="/autenticacion/ingresar")
21
22
23 def usuario_actual(
24     token: str = Depends(esquema_token),
25     sesion: Session = Depends(dar_sesion),
26 ) -> Usuario:

```

Figura 4.4. api/app/dependencias.py. Fuente: elaboración propia.

Para tener en cuenta

- Los cinco recursos: datos_pos, datos_trafico, datos_locatarios, auditoria y configuracion. Las cuatro operaciones: lectura, escritura, eliminacion y exportacion.
- Alcance por rol: el administrador tiene las veinte celdas; el gerente lee y exporta las cinco; operaciones lee y escribe tráfico; el locatario lee sus ventas y su tráfico.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

4.5 S2.4 · Colección de pruebas ejecutable de la API

En el plan: «Entregable de la semana 2: API de autenticación y usuarios funcionando, con la colección de pruebas para ejecutarla.»

Tarjeta del tablero: [#7 — S2.4 · Colección de pruebas ejecutable de la API](#)

Qué se hizo

1. Se armó conftest.py con una base SQLite en memoria, para poder correr la suite entera sin levantar Docker.
2. Se agregó conftest_postgres.py, que crea y descarta una base shopmetrics_prueba de verdad. Hizo falta porque SQLite no puede verificar las hipertablas de TimescaleDB, los uuid nativos ni las restricciones de unicidad.
3. La lista de migraciones del conftest estaba escrita a mano; en la semana 6 se descubrió que agregar una migración y olvidarse de sumarla ahí dejaba la base de prueba distinta de la real sin que nada fallara. Ahora se lee del directorio, igual que hace base/inicializar.sh.
4. Se escribió coleccion.http, la colección manual para recorrer la API desde el editor sin montar Postman.
5. Las pruebas corren con make probar; las del panel con make probar-panel; las dos más la nivelación, con make todo.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
api/pruebas/conftest.py	SQLite en memoria y los datos de prueba	bc3c304
api/pruebas/conftest_postgres.py	PostgreSQL real, migraciones leídas del directorio	c099013
api/pruebas/coleccion.http	La colección manual	5af3e86
api/pytest.ini	Marcas y configuración de pytest	07b9e3c
Makefile	probar, probar-panel, todo, nivelar	e660c3e

Tabla 4.6. Archivos de la tarea S2.4. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
--------------------	---------	------------

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

api/pruebas/test_autenticacion.py	7	CU-017-001
api/pruebas/test_permisos.py	10	CU-018-001
api/pruebas/test_postgres.py	11	Lo que SQLite no puede verificar
api/pruebas/test_integraciones.py	21	CU-002-001 y CU-003-001
api/pruebas/test_simuladores.py	12	Los simuladores y la forma del tráfico
api/pruebas/test_generador.py	6	La carga del centro de demostración
api/pruebas/test_metricas.py	22	CU-011-001 y CU-004-001
api/pruebas/test_salud.py	32	CU-001-001
api/pruebas/test_agregados.py	9	Los agregados continuos de TimescaleDB
api/pruebas/test_alertas.py	61	CU-006-001, CU-007-001 y CU-026-001

Tabla 4.7. Pruebas de la tarea S2.4. Fuente: elaboración propia.

Commits

[5af3e86](#) Semana 2: identidad y permisos

[07b9e3c](#) Probar contra PostgreSQL y nivelar código con documento

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				Fecha 22/09/2026
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

```

api/pruebas/conftest_postgres.py líneas 1 a 26 de 106
1  """Pruebas contra PostgreSQL de verdad, con el esquema que crean las migraciones.
2
3  Se usa una base aparte, `shopmetrics_prueba`, que se crea y se borra en cada
4  corrida, para no ensuciar la de desarrollo.
5  """
6  import os
7  import subprocess
8
9  import pytest
10 from fastapi.testclient import TestClient
11 from sqlalchemy import create_engine, text
12 from sqlalchemy.orm import sessionmaker
13
14 from app.base import dar_sesion
15 from app.main import api
16 from app.modelos import CentroAcceso, CentroComercial, Permiso, Rol, RolPermiso, Usuario, UsuarioRol
17 from app.seguridad import cifrar
18
19 CONTENEDOR = "shopmetrics-base"
20 BASE = "shopmetrics_prueba"
21 URL = f"postgresql+psycpg://shopmetrics:shopmetrics@localhost:5432/{BASE}"
22
23
24 def _psql(base, sql):
25     return subprocess.run(
26         ["docker", "exec", "-i", CONTENEDOR, "psql", "-U", "shopmetrics", "-d", base,

```

Figura 4.5. *api/pruebas/conftest_postgres.py*. Fuente: elaboración propia.

Para tener en cuenta

- Estado al cierre de la semana 6: 191 pruebas de la API y 76 del panel, todas en verde.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

5. Semana 3 · Ingesta de datos y generador de datos

5.1 Lo que pedía el plan de trabajo

El plan de trabajo compromete para esta semana lo siguiente. La figura reproduce la página tal como está en el documento presentado, sin reescribirla.

Semana 3 · Ingesta de datos y generador de datos

Sistema

- Alta y configuración de conectores POS con sus validaciones, incluida la del consentimiento (CU-002).
- Registro de dispositivos de conteo y recepción de sus eventos (CU-003).
- Generador de datos: un proceso externo que corre de forma continua produciendo transacciones de punto de venta y eventos de conteo. Es la fuente de datos del sistema durante todo el desarrollo y la demostración.

Negocio: Modelo de ingresos por línea de servicio y estacionalidad del retail.

	Entregable
Sistema	Alta de conector operativa y generador alimentando la base de forma continua.
Negocio	Modelo de ingresos armado y atado a la hoja de supuestos.

En la reunión se muestra: Dar de alta un conector y ver las transacciones llegando solas.

Figura 5.1. Semana 3 del plan de trabajo. Fuente: planificacion_de_proyecto.pdf, página 6.

Entregable comprometido: Alta de conector operativa y generador alimentando la base de forma continua.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

En la reunión se muestra: Dar de alta un conector y ver las transacciones llegando solas.

Punto de negocio de la semana: Modelo de ingresos por línea de servicio y estacionalidad del retail.

El punto de negocio no forma parte de este informe, que cubre únicamente la construcción del sistema.

5.2 S3.1 · Alta de conectores POS con validación contra el sistema externo

En el plan: «Alta y configuración de conectores POS con sus validaciones, incluida la del consentimiento (CU-002).»

Tarjeta del tablero: [#8 — S3.1 · Alta de conectores POS con validación contra el sistema externo](#)

Qué se hizo

1. Se implementó el alta con el orden que fija el diagrama de secuencia: primero se valida contra el sistema externo y recién después se guarda. Si la validación falla no queda nada escrito, ni siquiera un conector en estado caído. Es el curso alternativo CA-1.
2. Se escribió `cifrado.py` con Fernet (AES-128-CBC con HMAC-SHA256) para `api_key_enc`. La credencial nunca se escribe en claro y la API devuelve sólo los últimos cuatro caracteres enmascarados.
3. La clave de cifrado de credenciales es distinta de la de firma de tokens, a propósito: si se filtra una, la otra sigue protegiendo lo suyo.
4. Se implementó la validación de consentimiento del paso 4: sin consentimiento otorgado del locatario no hay alta, y el rechazo dice el motivo.
5. Se construyó el simulador de los siete POS (simuladores/pos/), cada uno con su cabecera de autenticación y su envoltorio de respuesta. Es el sistema externo contra el que el conector habla HTTP de verdad.
6. Se escribió `sincronizador.py`, la ingesta que el paso 6 manda arrancar. Normaliza los nombres de campo de cada proveedor a un modelo único: eso es el RF-05.
7. La sincronización trae sólo lo posterior al último registro de cada conector, así puede correr cada pocos minutos sin duplicar nada. Un conector que falla se marca como caído y no interrumpe a los demás.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

Archivos que quedaron escritos

Tabla 5.1. Archivos de la tarea S3.1. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Archivo de pruebas	Cuántas	Qué cubren
api/pruebas/test_integraciones.py	21	Levantar el simulador POS en un hilo, así el conector habla HTTP de verdad contra algo que autentica y que falla cuando tiene que fallar. Cubren el alta correcta, la credencial inválida (CA-1), la falta de consentimiento, el sistema desconocido, el conector duplicado, que la credencial no esté en claro en la base, que una credencial alterada no se descifre, y que el alta quede en auditoría.

Tabla 5.2. Pruebas de la tarea S3.1. Fuente: elaboración propia.

- [69a388e](#) Semana 3: conectores POS y dispositivos de conteo, con simulador propio
- [fa068b7](#) Semana 3: generador de datos y simulador de sensores de conteo

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

```

api/app/cifrado.py                                     líneas 1 a 26 de 62
1  """Cifrado de las credenciales de los sistemas externos.
2
3  El campo `api_key_enc` del diccionario de datos dice, textualmente, "Credencial
4  de API almacenada encriptada". Este modulo es el que lo cumple.
5
6  La diferencia con `seguridad.py` es importante: alla se cifran contraseñas, que
7  solo hay que **comparar**, y por eso se usa bcrypt, que es de una sola via.
8  Aca hay que **recuperar** la credencial para llamar al POS, asi que hace falta
9  cifrado simetrico reversible. Se usa Fernet, que es AES-128 en modo CBC con
10 HMAC-SHA256 para autenticar: si alguien toca el texto cifrado, al descifrar
11 falla en vez de devolver basura.
12
13 La clave sale del entorno. En desarrollo hay una fija para que el equipo levante
14 sin configurar nada; en produccion se pasa por variable de entorno y nunca entra
15 al repositorio.
16 """
17 from __future__ import annotations
18
19 import base64
20 import hashlib
21
22 from cryptography.fernet import Fernet, InvalidToken
23
24 from app.configuracion import ajustes
25

```

Figura 5.2. api/app/cifrado.py. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

```

api/app/dominios/integraciones/sincronizador.py      líneas 1 a 26 de 136
1  """Trae la facturacion de los conectores activos y la persiste.
2
3  Es la "ingesta automatizada de facturacion" que el paso 6 del CU-002-001 manda
4  arrancar cuando el conector queda activo. Cada sistema POS devuelve los datos
5  con sus propios nombres de campo, asi que lo primero que hace este modulo es
6  normalizarlos a un modelo unico: eso es el RF-05 del punto 10.4.2.
7
8  En la semana 4 esta misma funcion la va a llamar una tarea periodica (CU-001-001).
9  """
10 from __future__ import annotations
11
12 import uuid
13 from datetime import datetime, timedelta, timezone
14 from typing import Optional
15
16 import httpx
17 from sqlalchemy import select
18 from sqlalchemy.orm import Session
19
20 from app.cifrado import descifrar
21 from app.dominios.integraciones import servicio
22 from app.modelos import ConectorPos, RegistroPos
23
24 # El estado que el usuario fija a mano desde el paso 6 del CU-001-001. Se define
25 # aca y no se importa de `salud` para no cruzar los dos modulos: salud lo importa
26 # de aca.

```

Figura 5.3. api/app/dominios/integraciones/sincronizador.py. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A		Turno: Noche	Año: 2026
ShopMetrics					Fecha 22/09/2026

```

simuladores/pos/main.py                                     líneas 1 a 26 de 121
1  """Simulador de los sistemas de punto de venta.
2
3  Imita los siete POS que el informe lista como integrables. Cada uno autentica a
4  su manera y devuelve los datos con su propia forma, porque así es en la
5  realidad: no hay un estandar, y el conector de ShopMetrics tiene que saber
6  hablar con cada uno.
7
8  No escribe en la base de ShopMetrics: se deja consultar, como el sistema real.
9  """
10 from __future__ import annotations
11
12 from datetime import datetime, timedelta, timezone
13 from typing import Optional
14
15 from fastapi import FastAPI, Header, HTTPException, Query, status
16
17 from trafico import horas_entre, ventas
18
19 api = FastAPI(
20     title="Simulador de sistemas POS",
21     version="1.0.0",
22     description=(
23         "Imita los siete puntos de venta integrables de ShopMetrics. "
24         "Existe para que el conector se pruebe contra algo que se comporta como "
25         "el sistema real: autentica, responde con su propio formato y falla "
26         "cuando la credencial no sirve."),

```

Figura 5.4. simuladores/pos/main.py. Fuente: elaboración propia.

Para tener en cuenta

- Los siete sistemas: fiserv, bejerman, tango, geopagos, aper, maxirest y fudo. Cada uno nombra distinto lo que para nosotros es monto y cantidad; normalizar eso es el RF-05 del punto 10.4.2.
- Verificado contra PostgreSQL: cinco conectores trajeron 24 horas de facturación con ticket promedio coherente por rubro —gastronomía \$14, electrónica \$174—.

5.3 S3.2 · Registro de dispositivos IoT de conteo y recepción de sus eventos

En el plan: «Registro de dispositivos de conteo y recepción de sus eventos (CU-003).»

Tarjeta del tablero: [#9 — S3.2 · Registro de dispositivos IoT de conteo y recepción de sus eventos](#)

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Qué se hizo

1. Se implementó el alta del dispositivo y el endpoint que recibe sus conteos.
2. El endpoint de eventos no exige sesión de usuario: quien llama es el dispositivo, no una persona. La identificación va por el propio identificador del dispositivo, que tiene que existir y estar activo.
3. El esquema del evento tiene dos campos —marca de tiempo y conteo, que no puede ser negativo—. No hay lugar donde pudiera viajar una imagen aunque el dispositivo quisiera mandarla, que es exactamente el punto: se publican cantidades, nunca imágenes ni datos personales.
4. Se construyó el simulador de sensores (simuladores/iot/) como servicio aparte del de POS, porque el comportamiento es distinto: el POS se consulta, el sensor empuja.
5. El simulador llama al endpoint de ShopMetrics por su cuenta cada tanto, igual que haría uno real instalado en una zona. Puede cargar historia de un saque para tener datos desde el minuto uno de una demostración.
6. Se corrigieron dos errores del simulador: sys.path.insert hacía que la prueba cargara el simulador de POS en vez del de sensores, y el Dockerfile copiaba trafico.py fuera del WORKDIR.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
simuladores/iot/main.py	El sensor que empuja sus conteos	fa068b7
simuladores/iot/Dockerfile	La imagen del simulador de sensores	fa068b7
api/app/dominios/integraciones/rutas.py	Alta y recepción de eventos	c099013
simuladores/pos/trafico.py	La forma del retail que comparten los dos	69a388e

Tabla 5.3. Archivos de la tarea S3.2. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
--------------------	---------	------------

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A		Turno: Noche	Año: 2026
ShopMetrics					Fecha 22/09/2026

api/pruebas/test_simuladores.py	12	Corren contra una ShopMetrics real levantada en un hilo: el simulador empuja por HTTP de verdad y lo que llega se verifica leyendo evento_trafico. Cubren arrancar, arrancar dos veces el mismo sensor (409, porque sería contar doble), detener, detener uno que no estaba emitiendo (404), un intervalo fuera de rango (422), cargar historia, un dispositivo que ShopMetrics no conoce —donde el simulador informa cero en vez de mentir—, y que el lazo empuje solo.
api/pruebas/test_integraciones.py	21	El alta del dispositivo y el lote vacío

Tabla 5.4. Pruebas de la tarea S3.2. Fuente: elaboración propia.

Commits

[69a388e](#) Semana 3: conectores POS y dispositivos de conteo, con simulador propio

[fa068b7](#) Semana 3: generador de datos y simulador de sensores de conteo

simuladores/iot/main.py	líneas 1 a 26 de 138
<pre> 1 """Simulador de los sensores de conteo. 2 3 A diferencia del POS, que se consulta, el sensor **empuja**: cada tanto le manda 4 a ShopMetrics cuanta gente conto. Este servicio imita ese comportamiento. 5 6 No escribe en la base: llama al endpoint publico de ShopMetrics, igual que 7 haria un sensor real instalado en una zona. 8 """ 9 from __future__ import annotations 10 11 import asyncio 12 import os 13 import sys 14 from datetime import datetime, timedelta, timezone 15 from typing import Dict, Optional 16 17 import httpx 18 from fastapi import FastAPI, HTTPException, status 19 from pydantic import BaseModel, Field 20 21 # `trafico.py` define la forma del retail y lo comparten los dos simuladores. 22 # Se busca al lado (asi queda en la imagen) o en la carpeta del simulador de POS 23 # (asi queda al trabajar desde el repositorio). 24 _aqui = os.path.dirname(os.path.abspath(__file__)) 25 for _donde in (_aqui, os.path.join(_aqui, "..", "pos")): 26 if os.path.exists(os.path.join(_donde, "trafico.py")):</pre>	

Figura 5.5. simuladores/iot/main.py. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

```

simuladores/pos/trafico.py                                líneas 1 a 26 de 108
1  """La forma del trafico y de las ventas del retail.
2
3  No es ruido al azar: reproduce la forma real, que es lo que hace que los
4  indicadores de la semana 4 y las anomalias de la semana 7 tengan sentido.
5  """
6  from __future__ import annotations
7
8  import hashlib
9  import math
10 import random
11 from datetime import date, datetime, timedelta
12
13 # Cuanta gente circula en cada hora, como proporcion del pico del dia.
14 # Arranca flojo a las 10, sube al mediodia, baja a la siesta, pico de 17 a 20.
15 POR_HORA = {
16     10: 0.25, 11: 0.45, 12: 0.70, 13: 0.80, 14: 0.55, 15: 0.45,
17     16: 0.55, 17: 0.75, 18: 0.95, 19: 1.00, 20: 0.85, 21: 0.45,
18 }
19
20 # El sabado casi duplica a un martes. Lunes es 0.
21 POR_DIA = {0: 0.70, 1: 0.65, 2: 0.70, 3: 0.80, 4: 1.00, 5: 1.35, 6: 1.10}
22
23 # Los picos del retail que pide el punto de negocio de la semana 3.
24 def estacionalidad(dia: date) -> float:
25     if dia.month == 10 and 8 <= dia.day <= 21:      # Dia de la Madre
26         return 1.45

```

Figura 5.6. simuladores/pos/trafico.py. Fuente: elaboración propia.

Para tener en cuenta

- La forma del tráfico está probada, no supuesta: hay pruebas de que es reproducible para un mismo momento —si no, nada sería verificable—, de que con el centro cerrado no circula nadie, y de que la tarde del sábado mueve más que la apertura del lunes.

5.4 S3.3 · Generador de datos que alimenta la base de forma continua

En el plan: «Generador de datos: un proceso externo que corre de forma continua produciendo transacciones de punto de venta y eventos de conteo. Es la fuente de datos del sistema durante todo el desarrollo y la demostración.»

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Tarjeta del tablero: [#10 — S3.3 · Generador de datos que alimenta la base de forma continua](#)

Qué se hizo

1. Se escribió `generador/correr.py` con tres acciones: preparar arma el centro de demostración, correr deja las dos fuentes produciendo, estado dice qué hay cargado.
2. El generador no inventa datos por su cuenta: arma el escenario en ShopMetrics y después enciende los simuladores, que son los que imitan los sistemas reales.
3. Se escribió `generador/carga.py` para las zonas, locatarios, locales y consentimientos, que todavía no tienen endpoint propio —su ABM es de una semana posterior—. Todo lo que sí tiene endpoint pasa por la API.
4. La carga es idempotente: se corre cada vez que se levanta el entorno, así que volver a correrla no puede duplicar nada.
5. Se agregaron los comandos `make demo`, `make generar` y `make hay`.
6. Se corrigió la salida del lazo, que decía «0 registros» sin aclarar que entre hora y hora no hay nada nuevo que traer y eso es lo correcto, no una falla.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
generador/correr.py	Prepara el centro y enciende las dos fuentes	fa068b7
generador/carga.py	Zonas, locatarios, locales y consentimientos	fa068b7
Makefile	demo, generar, hay, simuladores	e660c3e
docker-compose.yml	Los dos simuladores como servicios	fa068b7

Tabla 5.5. Archivos de la tarea S3.3. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
api/pruebas/test_generador.py	6	Verifican que la carga cree zonas, locatarios, locales y consentimientos; que correrla dos veces no duplique nada; que cada locatario quede con consentimiento otorgado —sin eso el alta del conector se rechaza—; que los rubros cargados sean los que el simulador sabe manejar; y que acepte el identificador del centro como texto, que es como viaja desde la API.

Tabla 5.6. Pruebas de la tarea S3.3. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

Commits

[fa068b7](#) Semana 3: generador de datos y simulador de sensores de conteo

generador/correr.py líneas 1 a 26 de 183

```

1  """Generador de datos: arma un centro completo y lo deja produciendo.
2
3  Es el proceso externo que pide la semana 3 del plan. No inventa datos por su
4  cuenta: arma el escenario en ShopMetrics --zonas, locatarios, consentimientos,
5  conectores y dispositivos-- y despues deja corriendo las dos fuentes.
6
7      python3 generador/correr.py preparar      arma el centro de demostracion
8      python3 generador/correr.py correr        deja las dos fuentes produciendo
9      python3 generador/correr.py estado        que hay cargado ahora
10
11  Los datos salen de los simuladores, que imitan los sistemas reales. El
12  generador solo enciende el lazo.
13  """
14  from __future__ import annotations
15
16  import argparse
17  import os
18  import sys
19  import time
20  from datetime import datetime, timezone
21
22  import httpx
23
24  SHOPMETRICS = os.environ.get("SHOPMETRICS", "http://localhost:8000")
25  SIMULADOR_POS = os.environ.get("SIMULADOR_POS", "http://localhost:9001")
26  SIMULADOR_IOT = os.environ.get("SIMULADOR_IOT", "http://localhost:9002")

```

Figura 5.7. generador/correr.py. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

```

generador/carga.py                                     líneas 1 a 26 de 91
1  """Arma la estructura del centro de demostracion directo contra la base.
2
3  Las zonas y los locatarios todavia no tienen endpoint propio --el ABM es de una
4  semana posterior-- asi que el generador los carga por aca. Todo lo que si tiene
5  endpoint (conectores, dispositivos, eventos) pasa por la API, como corresponde.
6
7  Es idempotente: correrlo dos veces no duplica nada.
8  """
9  from __future__ import annotations
10
11  import os
12  import sys
13  import uuid
14  from datetime import datetime, timezone
15  from typing import List, Tuple
16
17  _api = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api")
18  if _api not in sys.path:
19      sys.path.insert(0, _api)
20
21  from sqlalchemy import func, select                                # noqa: E402
22  from app.base import Sesion                                       # noqa: E402
23  from app.modelos import (Consentimiento, EventoTrafico, Local,    # noqa: E402
24                          Locatario, RegistroPos, Zona)
25
26  # El consentimiento que el paso 4 del CU-002-001 exige antes de dar de alta el

```

Figura 5.8. generador/carga.py. Fuente: elaboración propia.

Para tener en cuenta

- Verificación contra PostgreSQL real, con la base, la API y los dos simuladores levantados: 5 conectores × 13 horas de apertura = 65 registros de facturación, y 3 sensores × 29 conteos = 87 eventos de tráfico sobre 48 horas.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

6. Semana 4 · Métricas del centro

6.1 Lo que pedía el plan de trabajo

El plan de trabajo compromete para esta semana lo siguiente. La figura reproduce la página tal como está en el documento presentado, sin reescribirla.

Semana 4 · Métricas del centro

Sistema

- Tarea periódica de sincronización y tablero de salud de los conectores (CU-001).
- Cálculo de los indicadores del centro: tráfico, conversión, ventas y vacancia (CU-011).
- Agregación de tráfico por zona y por hora sobre TimescaleDB (CU-004).

Negocio: Proyección de ventas a tres años y estrategia de penetración con su presupuesto de marketing.

	Entregable
Sistema	Endpoints de métricas devolviendo valores calculados sobre los datos del generador.
Negocio	Proyección de ventas a tres años y estrategia de penetración.

En la reunión se muestra: Los cuatro indicadores calculados desde la base y la proyección de ventas.

Figura 6.1. Semana 4 del plan de trabajo. Fuente: planificacion_de_proyecto.pdf, página 6.

Entregable comprometido: Endpoints de métricas devolviendo valores calculados sobre los datos del generador.

En la reunión se muestra: Los cuatro indicadores calculados desde la base y la proyección de ventas.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Punto de negocio de la semana: Proyección de ventas a tres años y estrategia de penetración con su presupuesto de marketing.

El punto de negocio no forma parte de este informe, que cubre únicamente la construcción del sistema.

6.2 S4.1 · Sincronización periódica y tablero de salud de los conectores

En el plan: «Tarea periódica de sincronización y tablero de salud de los conectores (CU-001).»

Tarjeta del tablero: [#11 — S4.1 · Sincronización periódica y tablero de salud de los conectores](#)

Qué se hizo

1. Se implementó el tablero con las dos secciones del paso 3 —conectores POS y dispositivos IoT— y semáforo de tres colores.
2. La fecha de la última sincronización no se guarda en ningún campo: sale del máximo de la propia serie, como marca el caso de uso. Un campo que dijera «última sincronización» podría desincronizarse de los datos que dice describir; el máximo de la serie no puede.
3. La tolerancia del semáforo depende de qué integración sea: el POS factura por hora —2 h y 8 h—, el sensor emite cada pocos minutos —30 min y 2 h—.
4. Se implementaron las tres acciones correctivas del paso 6, cada una registrada en auditoría: reintentar conexión, reiniciar conector y suspender la ingesta.
5. Se agregó el estado suspendido, que es el curso alternativo CA-2: el conector sigue existiendo y el sincronizador lo saltea.
6. La latencia del detalle se mide en el momento —el ida y vuelta real contra el sistema externo— y no se guarda, que es lo único consistente con lo que pide el paso 5.
7. Se escribió la tarea periódica (tareas.py), que sincroniza, después revisa y al final evalúa las reglas. El orden importa: revisar antes de sincronizar marcaría caído a un conector que acaba de traer datos en esa misma vuelta.
8. Se corrigió una oscilación: dejando la tarea corriendo con el simulador apagado, cada vuelta informaba cinco cambios de estado. La revisión daba por recuperado a un conector caído si tenía facturación reciente, pero un conector caído sigue teniendo la

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

facturación de *antes* del corte. Ahora la revisión sólo degrada; recuperar exige una consulta que efectivamente anduvo.

9. El sincronizador pasó a reintentar los caídos. Sin eso, un conector que fallaba una vez quedaba caído hasta que alguien apretara un botón.

10. Se hizo visible el registro de la tarea: no tenía handler bajo uvicorn, así que trabajaba en silencio y no había forma de saber desde afuera si seguía viva.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
api/app/dominios/integraciones/salud.py	Semáforo, tablero, detalle y revisión	ba7dfcc
api/app/dominios/integraciones/consultas_salud.py	Las consultas que no escriben	e333212
api/app/tareas.py	La tarea periódica	c099013
api/app/dominios/integraciones/rutas.py	salud, detalle, acción y revisar	c099013

Tabla 6.1. Archivos de la tarea S4.1. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
api/pruebas/test_salud.py	32	El semáforo en sus tres colores y en las dos clases de integración; que el estado caído mande sobre la fecha; que suspendido salga en amarillo; que la última sincronización salga de la serie; las tres acciones correctivas con su auditoría; que una acción que no resuelve deje el conector caído (CA-1); que suspender no dé de baja (CA-2); que la revisión no rescite por datos viejos; que el sincronizador reintente los caídos; y que la vuelta informe las recuperaciones.

Tabla 6.2. Pruebas de la tarea S4.1. Fuente: elaboración propia.

Commits

[e333212](#) Semana 4: salud de las integraciones, indicadores del centro y agregados

[ba7dfcc](#) Corregir la oscilacion del estado de los conectores

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			Fecha 22/09/2026
	Sede: Centro	Comisión: 5° A	Turno: Noche	
ShopMetrics				

```

api/app/dominios/integraciones/salud.py                                líneas 1 a 26 de 204
1  """Estado de salud de las integraciones.
2
3  CU-001-001 Monitorear el estado de las integraciones
4
5  El semaforo tiene tres colores porque el caso de uso los pide asi --verde,
6  amarillo, rojo-- y cada uno significa algo distinto: verde es que la integracion
7  trajo datos hace poco, amarillo es que hace rato que no trae pero todavia no hay
8  motivo para darla por perdida, y rojo es que fallo o que lleva demasiado callada.
9
10 La fecha de la ultima sincronizacion **no se guarda**: sale de la propia serie,
11 como marca el paso 3 del caso de uso. Eso evita tener un campo que se pueda
12 desincronizar de los datos que dice describir.
13 """
14 from __future__ import annotations
15
16 import time
17 import uuid
18 from datetime import datetime, timedelta, timezone
19 from typing import List, Optional
20
21 from sqlalchemy import select
22 from sqlalchemy.orm import Session
23
24 from app.cifrado import descifrar
25 from app.dominios.integraciones import consultas_salud as q
26 from app.dominios.integraciones import servicio

```

Figura 6.2. api/app/dominios/integraciones/salud.py. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

```

api/app/tareas.py                                     líneas 1 a 26 de 103
1  """La tarea periodica que mantiene las integraciones al dia.
2
3  CU-001-001 paso 2: el sistema consulta el estado actual de todos los conectores
4  y dispositivos. Para que ese estado signifique algo tiene que actualizarse solo,
5  aunque nadie abra el tablero.
6
7  Cada vuelta hace dos cosas, en este orden:
8
9      1. Sincroniza los conectores activos, trayendo lo que falte.
10     2. Revisa todas las integraciones y deja su `estado` al dia.
11     3. Evalua las reglas de alerta contra los datos recién traídos.
12
13  El orden importa. Revisar despues de sincronizar evita marcar como caído a un
14  conector que acaba de traer datos en esta misma vuelta; evaluar al final hace
15  que las reglas vean lo ultimo y no lo de la vuelta anterior.
16
17  Se enciende poniendo `SEGUNDOS_ENTRE_SINCRONIZACIONES` en el entorno. Apagada
18  por omision, así las pruebas no arrancan un lazo de fondo.
19  """
20  from __future__ import annotations
21
22  import asyncio
23  import logging
24
25  from app.base import Sesion
26  from app.configuracion import ajustes

```

Figura 6.3. api/app/tareas.py. Fuente: elaboración propia.

Para tener en cuenta

- Verificación que pedía la tarjeta: apagando el simulador POS, los cinco conectores pasaron a rojo con el motivo; encendiéndolo, volvieron a activo solos. Con la tarea corriendo, una sola transición en cada sentido por conector.
- El paso 5 pide «latencia promedio» y a la vez dice que no se almacena. Un promedio es de varias mediciones. Quedó como DT-13.

Deuda técnica detectada

- [Tarjeta #56 del tablero](#)

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			Fecha 22/09/2026
	Sede: Centro	Comisión: 5° A	Turno: Noche	
ShopMetrics				

6.3 S4.2 · Cálculo de los indicadores del centro

En el plan: «Cálculo de los indicadores del centro: tráfico, conversión, ventas y vacancia (CU-011).»

Tarjeta del tablero: [#12 — S4.2 · Cálculo de los indicadores del centro](#)

Qué se hizo

1. Se creó el dominio metricas con tres capas: consultas.py habla con la base, servicio.py decide, rutas.py traduce a HTTP.
2. Ningún indicador se guarda: los cuatro se calculan sobre las series en el momento de la consulta, como fija el caso de uso, que los marca «calculado, no se almacena».
3. La conversión se agrega por franja horaria, nunca persona por persona: se suman los tickets de una hora y se dividen por las entradas de esa misma hora. Eso la hace anónima por construcción, y es lo que sostiene el compromiso de privacidad del trabajo.
4. Sólo entran las horas que tienen las dos fuentes. Una hora con tráfico pero sin ventas hundiría la conversión por un problema de integración, no del negocio.
5. Se implementaron los tres períodos del paso 2 y la variación contra el período anterior. Cuando no hay contra qué comparar, la variación es nula y se dice: un cero antes no significa «creció infinito».
6. El curso alternativo CA-1 viaja dentro de cada indicador: parcial, el aviso y la fuente afectada. Un indicador que calla lo que no sabe es peor que uno que no está.
7. Se corrigió la semilla, que nombraba indicadores distintos de los cuatro del informe.
8. Se corrigió un error que fallaba en silencio: las consultas crudas comparaban los UUID pasándolos como texto. Sobre PostgreSQL andaba; sobre SQLite, que los guarda como treinta y dos caracteres hexadecimales sin guiones, devolvía cero filas y no un error, así que cada indicador salía en cero sin que nada se rompiera.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
api/app/dominios/metricas/servicio.py	Los cuatro KPIs, períodos y variación	c099013
api/app/dominios/metricas/consultas.py	Las consultas, con el tipo declarado	e333212
api/app/dominios/metricas/rutas.py	tablero, serie, zonas y detalle	e333212

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

base/semilla/001_datos_base.sql	Los cuatro indicadores del informe	c099013
---	------------------------------------	---------

Tabla 6.3. Archivos de la tarea S4.2. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
api/pruebas/test_metricas.py	22	Los datos se siembran en horas conocidas para que cada indicador tenga un valor calculable a mano: 600 personas, \$6.000 y 10 % de conversión. Si el endpoint devolviera una constante, esos números no saldrían. Cubren la agregación horaria, que una hora sin ventas no hunda la conversión, el CA-1, la variación, los tres períodos, el drill-down y los permisos.

Tabla 6.4. Pruebas de la tarea S4.2. Fuente: elaboración propia.

Commits

[e333212](#) Semana 4: salud de las integraciones, indicadores del centro y agregados

api/app/dominios/metricas/servicio.py	líneas 1 a 26 de 290
<pre> 1 """Los cuatro indicadores del centro. 2 3 CU-011-001 Tablero de indicadores del centro 4 CU-004-001 Vista de mapa del centro 5 6 Ningun indicador se guarda: todos se calculan sobre las series en el momento de 7 la consulta, como fija el caso de uso. La tabla `indicador` guarda el catalogo 8 --nombre y unidad-- para que las reglas de alerta puedan apuntarle, no valores. 9 """ 10 from __future__ import annotations 11 12 import uuid 13 from datetime import datetime, timedelta, timezone 14 from typing import List, Optional, Tuple 15 16 from sqlalchemy import select 17 from sqlalchemy.orm import Session 18 19 from app.dominios.metricas import consultas 20 from app.modelos import DispositivoIot, Local, Locatario, Zona 21 22 # Los tres periodos del paso 2 del CU-011-001. 23 PERIODOS = {"hoy": timedelta(days=1), "7dias": timedelta(days=7), "30dias": timedelta(days=30)} 24 25 # Los cuatro indicadores clave que el informe enumera para el tablero. 26 TRAFICO, CONVERSION, VENTAS, VACANCIA = "trafico", "conversion", "ventas", "vacancia" </pre>	

Figura 6.4. api/app/dominios/metricas/servicio.py. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

```

api/app/dominios/metricas/consultas.py                                líneas 1 a 26 de 125
1  """Las consultas sobre las series temporales.
2
3  Todo lo que el panel muestra se calcula al momento: ningun indicador se guarda.
4  Es lo que fija el CU-011-001, que marca los cuatro KPIs como «calculado, no se
5  almacena».
6
7  Sobre PostgreSQL las consultas leen los **agregados continuos** de TimescaleDB
8  (`trafico_por_hora`, `ventas_por_hora`), que mantienen el resultado por hora ya
9  calculado. Sobre SQLite --que es donde corre la mayor parte de las pruebas-- no
10 hay agregados, así que la misma consulta se arma sobre la tabla cruda. El
11 resultado tiene que ser identico en las dos: hay una prueba que lo compara.
12 """
13 from __future__ import annotations
14
15 import uuid
16 from datetime import datetime
17 from typing import List, Optional
18
19 from sqlalchemy import Uuid, bindparam, text
20 from sqlalchemy.orm import Session
21
22
23 def es_postgres(sesion: Session) -> bool:
24     return sesion.get_bind().dialect.name == "postgresql"
25

```

Figura 6.5. api/app/dominios/metricas/consultas.py. Fuente: elaboración propia.

Para tener en cuenta

- Verificación contra la base real: 16.427 personas, 14,49 % de conversión y \$44.616 de facturación sobre los datos del generador.
- La vacancia es el único de los cuatro que el modelo no sostiene: local.locatario_id es obligatorio, así que un local vacante no se puede representar. Quedó como DT-12 y el indicador viaja marcado como parcial.

Deuda técnica detectada

- [Tarjeta #55 del tablero](#)

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

6.4 S4.3 · Agregación de tráfico por zona y por hora sobre TimescaleDB

En el plan: «Agregación de tráfico por zona y por hora sobre TimescaleDB (CU-004).»

Tarjeta del tablero: [#13 — S4.3 · Agregación de tráfico por zona y por hora sobre TimescaleDB](#)

Qué se hizo

1. Se escribió 003_agregados.sql con dos agregados continuos: trafico_por_hora y ventas_por_hora. Mantienen el resultado por hora ya calculado y lo actualizan sólo sobre los trozos que cambiaron.
2. Se agregó la política de refresco: cada media hora sobre la última semana, con un desfase de una hora para que la hora en curso termine de llenarse.
3. El refresco en tiempo real quedó declarado explícito, y esto casi pasa inadvertido: desde TimescaleDB 2.13 materialized_only viene en true por omisión, así que la vista devuelve sólo lo ya materializado. La primera consulta después de crear las vistas devolvió cero filas con datos cargados; en la demo la hora en curso habría salido vacía.
4. Sobre SQLite no hay agregados, así que la misma consulta se arma sobre la tabla cruda con strftime. Son dos consultas distintas que tienen que dar el mismo número.
5. Se implementó el mapa del centro con el semáforo de cobertura del paso 4, que distingue cuatro situaciones porque no son lo mismo.
6. Una zona sin cobertura devuelve personas_por_m2 nulo y no cero: un cero se confundiría con «no pasó nadie». Es el curso alternativo CA-1.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
base/migraciones/003_agregados.sql	Los dos agregados continuos	e333212
api/app/dominios/metricas/consultas.py	Agregado en PostgreSQL, tabla cruda en SQLite	e333212
api/app/dominios/metricas/servicio.py	Mapa del centro y detalle de zona	c099013

Tabla 6.5. Archivos de la tarea S4.3. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
api/pruebas/test_agregados.py	9	Sólo contra PostgreSQL, porque esto SQLite no lo puede verificar: que los agregados existan, que tengan el refresco en tiempo real encendido —para que un cambio de versión no lo reintroduzca en silencio—, que tengan política de refresco, que vean lo todavía no materializado, que agrupen de verdad por hora, y que el tablero dé el mismo número sobre PostgreSQL que sobre SQLite.

Tabla 6.6. Pruebas de la tarea S4.3. Fuente: elaboración propia.

Commits

[e333212](#) Semana 4: salud de las integraciones, indicadores del centro y agregados

base/migraciones/003_agregados.sql	líneas 1 a 26 de 60
<pre> 1 -- Agregados continuos de TimescaleDB. 2 -- 3 -- El panel consulta el trafico y la facturacion por hora todo el tiempo. Sin 4 -- agregacion previa, cada consulta recorrería la tabla de eventos entera. Un 5 -- agregado continuo mantiene el resultado por hora ya calculado y lo actualiza 6 -- solo sobre los trozos que cambiaron. 7 -- 8 -- Tienen refresco en tiempo real: lo que todavía no se materializo se calcula 9 -- al vuelo sobre los datos crudos, así que una consulta nunca devuelve una hora 10 -- incompleta por estar esperando el refresco. 11 -- 12 -- CU-004-001 Vista de mapa del centro (trafico por zona y franja horaria) 13 -- CU-011-001 Tablero de indicadores del centro 14 -- 15 -- Trafico por dispositivo y por hora. 16 CREATE MATERIALIZED VIEW IF NOT EXISTS trafico_por_hora 17 WITH (timescaledb.continuous) AS 18 SELECT 19 dispositivo_id, 20 time_bucket(INTERVAL '1 hour', ts) AS hora, 21 SUM(conteo)::BIGINT AS personas, 22 COUNT(*)::BIGINT AS mediciones 23 FROM evento_trafico 24 GROUP BY dispositivo_id, hora 25 WITH NO DATA;</pre>	

Figura 6.6. base/migraciones/003_agregados.sql. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

Para tener en cuenta

- Verificación contra la base real: patio de comidas 5.395 personas sobre 900 m² (5,99 por m²), planta baja 5.610 sobre 1.800 (3,12). El patio tiene casi el doble de densidad sobre un tercio de la superficie, que es lo que se espera de esa zona.

Deuda técnica detectada

- [Tarjeta #58 del tablero](#)

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			Fecha 22/09/2026
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				

7. Semana 5 · Panel del administrador

7.1 Lo que pedía el plan de trabajo

El plan de trabajo compromete para esta semana lo siguiente. La figura reproduce la página tal como está en el documento presentado, sin reescribirla.

Semana 5 · Panel del administrador

Sistema

- Frontend en React con Next.js: estructura, ingreso y navegación por rol.
- Dashboard consumiendo los endpoints reales de métricas.
- Diseño responsive desde el inicio, para que las mismas pantallas sirvan en escritorio y en celular.

Negocio: Costos fijos, variables y operativos, separados y costeados.

	Entregable
Sistema	Panel del administrador mostrando los indicadores y el tráfico por hora traídos de la API.
Negocio	Cuadro de costos fijos, variables y operativos.

Figura 7.1. Semana 5 del plan de trabajo. Fuente: planificacion_de_proyecto.pdf, página 7.

Entregable comprometido: Panel del administrador mostrando los indicadores y el tráfico por hora traídos de la API.

Punto de negocio de la semana: Costos fijos, variables y operativos, separados y costeados.

El punto de negocio no forma parte de este informe, que cubre únicamente la construcción del sistema.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

7.2 S5.1 · Frontend en Next.js: estructura, ingreso y navegación por rol

En el plan: «Frontend en React con Next.js: estructura, ingreso y navegación por rol.»

Tarjeta del tablero: [#14 — S5.1 · Frontend en Next.js: estructura, ingreso y navegación por rol](#)

Qué se hizo

1. Se armó web/ con Next.js 15 y TypeScript, en cuatro carpetas: app/ una por ruta, componentes/, servicios/ y pruebas/.
2. Los nombres van en castellano, como el resto del proyecto. Las excepciones son las que la herramienta impone: page.tsx, layout.tsx y next.config.ts.
3. Se declararon los tipos del contrato una sola vez en servicios/tipos.ts, reflejando los esquemas de Pydantic. Un cambio en la API rompe la compilación en vez de romperse en silencio delante del usuario.
4. El token se adjunta en un solo lugar, en servicios/api.ts. No hay forma de olvidárselo al escribir una pantalla nueva.
5. La sesión vive en sessionStorage y no en localStorage, a propósito: se borra al cerrar la pestaña. Para un sistema que muestra facturación de terceros, que la sesión no sobreviva al navegador cerrado es lo que corresponde.
6. El destino después de entrar no lo decide la pantalla: viene en el campo panel de la respuesta. Decidirlo en el frente duplicaría la regla del paso 5 y las dos copias se irían separando.
7. El menú se filtra por la lista permisos, con los nombres exactos de la matriz del CU-018-001. Es cortesía, no seguridad: quien decide sigue siendo el backend, que devuelve 403.
8. Se corrigió el manejo del 401, que una prueba destapó: el ingreso fallido mostraba «La sesión venció», cuando nunca hubo sesión que venciera. Ahora se distingue el token rechazado de las credenciales que no sirven.
9. Se agregó CORS a la API. El panel corre en otro puerto, así que para el navegador es otro origen y el pedido no llegaba a salir: lo cortaba el propio navegador. Se enumeran los orígenes permitidos, no se abre a cualquiera.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
web/app/ingresar/page.tsx	La pantalla de ingreso (CU-017-001)	e660c3e
web/app/panel/layout.tsx	La guardia de sesión y el menú	e660c3e
web/componentes/Menu.tsx	El menú filtrado por permisos	c099013
web/servicios/api.ts	El cliente: token, 401 y errores	c099013
web/servicios/sesion.ts	Dónde vive la sesión y qué puede el usuario	e660c3e
web/servicios/tipos.ts	Los contratos de la API	c099013
web/componentes/usarPedido.ts	Los tres estados de un pedido	e660c3e
web/app/layout.tsx	El viewport, sin el cual no hay responsive	e660c3e
api/app/main.py	El middleware de CORS	e660c3e
web/README.md	Cómo está armado el panel y por qué	e660c3e

Tabla 7.1. Archivos de la tarea S5.1. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
web/pruebas/sesion.test.ts	11	Dónde vive la sesión, que se descarte si está corrupta, que no esté en localStorage, y los permisos por los dos lados.
web/pruebas/api.test.ts	8	Que el token viaje en cada pedido, que el 401 con sesión la borre, que el 401 sin sesión pase el motivo de la API, y que no llegar al servidor se distinga de que el servidor diga que no.
web/pruebas/pantallas.test.tsx	18	El ingreso con su destino según rol, el menú por permisos, y el tablero.

Tabla 7.2. Pruebas de la tarea S5.1. Fuente: elaboración propia.

Commits

[e660c3e](#) Semana 5: el panel en Next.js, consumiendo la API real

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

```

web/servicios/api.ts                                     líneas 1 a 26 de 190
1  /* El cliente de la API.
2  *
3  * Todo pedido pasa por aca, por dos motivos: para que el token se adjunte en
4  * un solo lugar --y no haya forma de olvidarselo-- y para que el 401 se
5  * maneje igual en todas las pantallas.
6  */
7  import { borrar, sesionGuardada } from "../sesion";
8  import type {
9    AlertaVista, Bandeja, CatalogoDeReglas, DetalleDeAlerta, Periodo, ReglaVista,
10   Serie, Sesion, Tablero, TableroDeSalud, ZonaDelMapa,
11 } from "../tipos";
12
13 const API = process.env.API ?? "http://localhost:8000";
14
15 /** Un error de la API, con el codigo para poder distinguirlos. */
16 export class ErrorDeApi extends Error {
17   codigo: number;
18
19   constructor(codigo: number, mensaje: string) {
20     super(mensaje);
21     this.codigo = codigo;
22     this.name = "ErrorDeApi";
23   }
24 }
25
26 /**

```

Figura 7.2. web/servicios/api.ts. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

```

web/componentes/Menu.tsx                                     líneas 1 a 26 de 83
1  /* El menu lateral, armado segun lo que el rol puede ver.
2  *
3  * CU-018-001: la matriz de permisos decide que aparece. Esconder lo que el rol
4  * no puede ver es cortesía, no seguridad: quien decide sigue siendo el backend,
5  * que devuelve 403. Aca se evita ofrecer una puerta que no abre.
6  */
7  "use client";
8
9  import Link from "next/link";
10 import { usePathname, useRouter } from "next/navigation";
11 import { borrar, puede } from "@servicios/sesion";
12 import type { Sesion } from "@servicios/tipos";
13
14 type Opcion = {
15   ruta: string;
16   texto: string;
17   recurso: string;
18   operacion: string;
19   grupo?: string;
20 };
21
22 /* Cada opcion declara que permiso necesita, con los nombres exactos de la
23   matriz del paso 4 del CU-018-001. Las secciones que todavia no se
24   construyeron no figuran: se agregan en la semana que las construye. */
25 const OPCIONES: Opcion[] = [
26   { ruta: "/panel", texto: "Inicio", recurso: "datos_trafico", operacion: "lectura" },

```

Figura 7.3. web/componentes/Menu.tsx. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

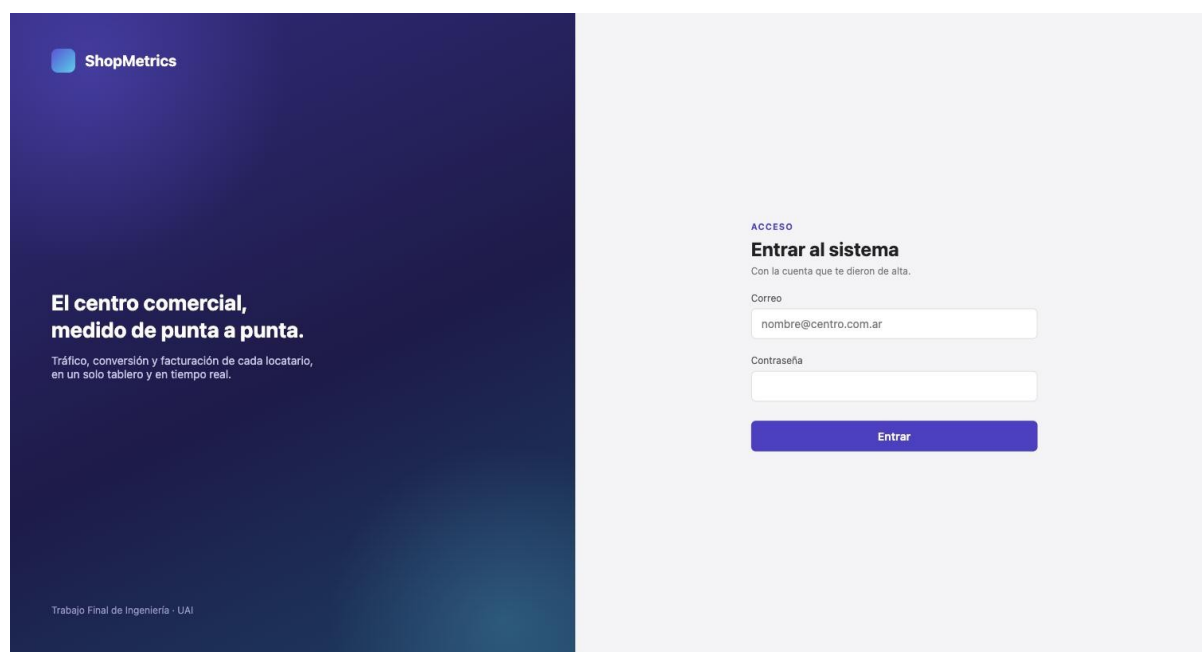


Figura 7.4. Pantalla de ingreso. El destino posterior lo decide el backend, no esta pantalla. Fuente: elaboración propia.

Para tener en cuenta

- El diseño sale del prototipo publicado en <https://javieravellaneda.github.io/shopmetrics-prototipo/> : la misma paleta, el mismo menú lateral oscuro y las mismas cajas blancas.

7.3 S5.2 · Panel del administrador consumiendo los endpoints reales

En el plan: «Dashboard consumiendo los endpoints reales de métricas.»

Tarjeta del tablero: [#15 — S5.2 · Panel del administrador consumiendo los endpoints reales](#)

Qué se hizo

1. Se armó la pantalla de inicio con los cuatro indicadores y la serie por hora, todo traído de la API: no hay ningún número escrito en la pantalla.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A		Turno: Noche	Año: 2026
ShopMetrics					Fecha 22/09/2026

2. El aviso de dato parcial llega hasta la caja del indicador, así que el curso alternativo CA-1 se ve donde el usuario lo necesita.
3. Se implementó el drill-down del paso 7: el mismo gráfico, cambiando de indicador.
4. Se escribió usarPedido, que descarta la respuesta de un pedido que quedó viejo. Si el usuario cambia de período antes de que vuelva el anterior, la respuesta vieja puede llegar después y pisar a la buena.
5. Se armaron las pantallas de mapa del centro y de integraciones contra los mismos endpoints.
6. Se corrigió el gráfico, que salía con las barras planas. La altura de cada barra es un porcentaje del máximo, y un porcentaje sólo resuelve contra una altura definida: el contenedor tenía min-height en vez de height, así que todas las barras medían lo que medía su etiqueta. Compilaba, pasaba el chequeo de tipos y se veía roto.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
web/app/panel/page.tsx	El tablero de indicadores	e660c3e
web/componentes/Indicador.tsx	La caja de KPI con su aviso	e660c3e
web/componentes/GraficoDeBarras.tsx	La serie por hora, sin librería	e660c3e
web/componentes/SelectorDePeriodo.tsx	Los tres períodos	e660c3e
web/app/panel/zonas/page.tsx	El mapa del centro	e660c3e
web/app/panel/integraciones/page.tsx	Salud y acciones correctivas	c099013
web/componentes/Semaforo.tsx	El color nunca va solo	e660c3e

Tabla 7.3. Archivos de la tarea S5.2. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
web/pruebas/componentes.test.tsx	16	Cómo se escribe cada unidad, la variación, el aviso de dato parcial, y las alturas del gráfico: 50 %, 100 % y 25 % para una serie de 100, 200 y 50.
web/pruebas/pantallas.test.tsx	18	Incluye una prueba de que no hay números escritos: se hace responder a la API con otro valor y se verifica que la pantalla muestre ese otro.

Tabla 7.4. Pruebas de la tarea S5.2. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Commits

[e660c3e](#) Semana 5: el panel en Next.js, consumiendo la API real

```

web/app/panel/page.tsx                                     líneas 1 a 26 de 89
1  /* CU-011-001 Tablero de indicadores del centro.
2  *
3  * Los cuatro indicadores y la serie por hora vienen de la API: no hay ningun
4  * numero escrito en esta pantalla. Cambiar algo en la base se tiene que ver
5  * aca, que es como se verifica esta parte.
6  */
7  "use client";
8
9  import { useState } from "react";
10 import GraficoDeBarras from "@componentes/GraficoDeBarras";
11 import Indicador from "@componentes/Indicador";
12 import SelectorDePeriodo from "@componentes/SelectorDePeriodo";
13 import { usarPedido } from "@componentes/usarPedido";
14 import { serie, tablero } from "@servicios/api";
15 import { centroActual, sesionGuardada } from "@servicios/sesion";
16 import type { Periodo } from "@servicios/tipos";
17
18 /* Que se dibuja abajo de los indicadores, segun cual se este mirando. */
19 const UNIDAD_DEL_GRAFICO = {
20   trafico: "personas",
21   ventas: "pesos",
22   conversion: "porcentaje",
23 } as const;
24
25 type Dibujable = keyof typeof UNIDAD_DEL_GRAFICO;

```

Figura 7.5. web/app/panel/page.tsx. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

```

web/componentes/GraficoDeBarras.tsx                                líneas 1 a 26 de 67
1  /* El grafico por hora del tablero y del detalle de zona.
2  *
3  * Sin libreria: las barras son divs con la altura en porcentaje del maximo.
4  * Para una serie de dos dias por hora no hace falta mas, y es una dependencia
5  * menos que mantener.
6  */
7  import type { PuntoDeSerie } from "@servicios/tipos";
8  import { conPuntos } from "../Indicador";
9
10 type Props = {
11   puntos: PuntoDeSerie[];
12   /** Como se escribe cada valor en el eje y en el globo de ayuda. */
13   unidad?: "personas" | "pesos" | "porcentaje";
14 };
15
16 /** La hora, sin la fecha: en el eje no entra y no aporta. */
17 function soloLaHora(marca: string): string {
18   const coincide = marca.match(/[T ](\d{2}):/);
19   return coincide ? `${coincide[1]}h` : marca.slice(-5);
20 }
21
22 function escribir(valor: number, unidad: Props["unidad"]): string {
23   if (unidad === "pesos") return `$ ${conPuntos(valor)}`;
24   if (unidad === "porcentaje") return `${conPuntos(valor, 2)} %`;
25   return conPuntos(valor);
26 }

```

Figura 7.6. web/componentes/GraficoDeBarras.tsx. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

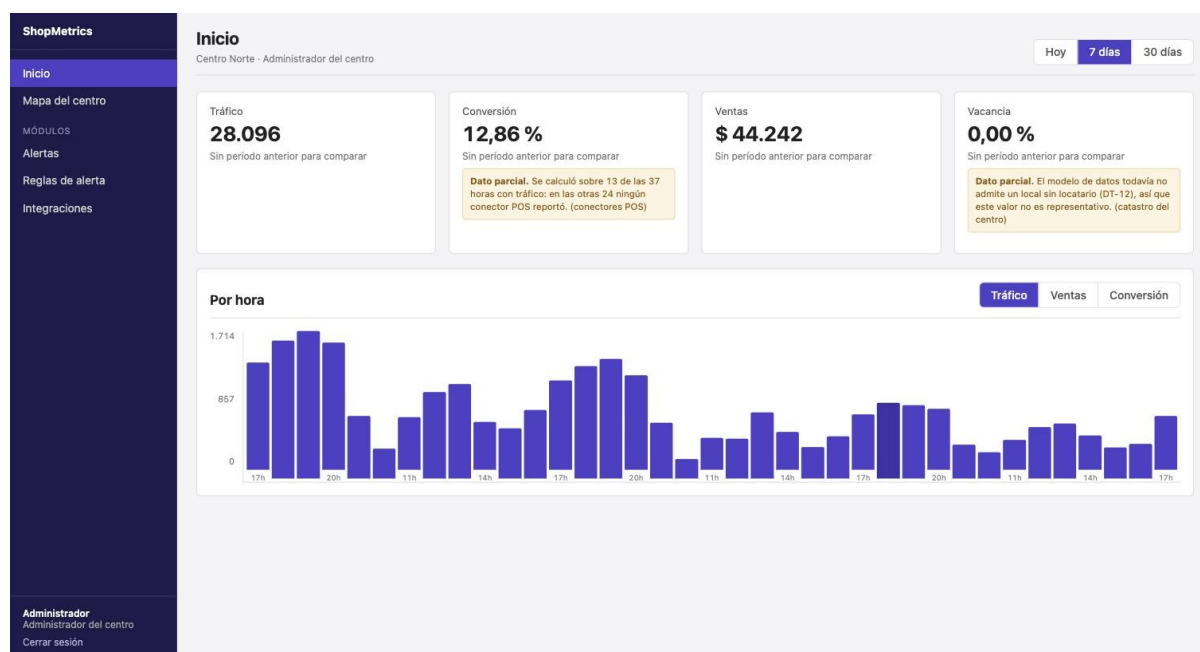


Figura 7.7. Tablero de indicadores. Los cuatro valores y la serie por hora vienen de la API. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				Fecha 22/09/2026
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

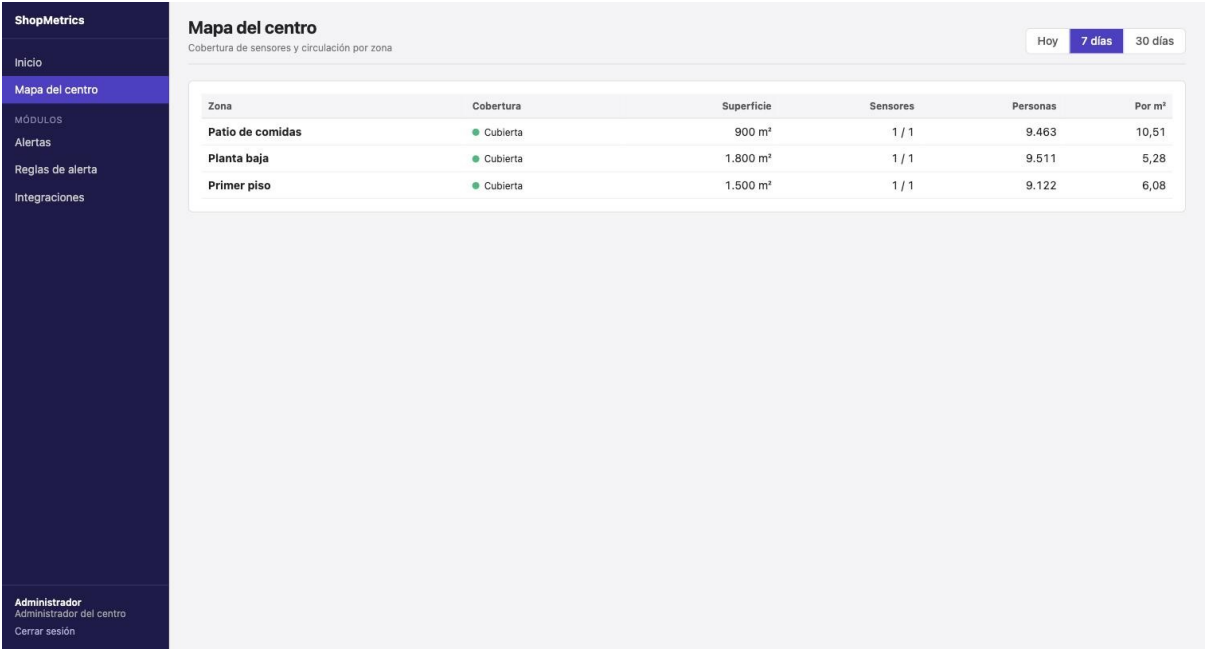


Figura 7.8. Mapa del centro, con el semáforo de cobertura por zona. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				

Fecha
22/09/2026

ShopMetrics Inicio Mapa del centro MÓDULOS Alertas Reglas de alerta Integraciones Administrador Administrador del centro Cerrar sesión	Integraciones 8 en línea · 0 con demora · 0 caídas				
	Conectores POS				
	Locatario	Sistema	Estado	Última sincronización	Última hora Acción
	Sport Central	bejerman	En línea	22/9, 02:00 p. m.	1 Elegir...
	Indumentaria K	fiserv	En línea	22/9, 02:00 p. m.	1 Elegir...
	Café Mundo	fudo	En línea	22/9, 02:00 p. m.	1 Elegir...
	Optic Plus	geopagos	En línea	22/9, 02:00 p. m.	1 Elegir...
	Tech Store	tango	En línea	22/9, 02:00 p. m.	1 Elegir...
	Dispositivos de conteo				
	Zona	Modelo	Estado	Último evento	Última hora
	Planta baja	V-Count Nano	En línea	22/9, 02:53 p. m.	4
	Primer piso	V-Count Nano	En línea	22/9, 02:53 p. m.	4
	Patio de comidas	V-Count Nano	En línea	22/9, 02:53 p. m.	4

Figura 7.9. Salud de las integraciones, con la última sincronización de cada fuente. Fuente: elaboración propia.

Para tener en cuenta

- Verificación que pedía la tarjeta —cambiar algo en la base y ver que el panel lo refleje—: apagando el simulador POS y forzando una sincronización, la pantalla de integraciones pasó de «8 en línea · 0 caídas» a «3 en línea · 5 caídas», con los cinco en rojo y el motivo debajo de cada uno.

7.4 S5.3 · Diseño responsive desde el inicio

En el plan: «Diseño responsive desde el inicio, para que las mismas pantallas sirvan en escritorio y en celular.»

Tarjeta del tablero: [#16 — S5.3 · Diseño responsive desde el inicio](#)

Qué se hizo

1. Se escribió una sola hoja de estilos con la paleta del prototipo como variables CSS, y los puntos de quiebre desde el arranque.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

2. Abajo de 820 px el menú lateral pasa a ser una barra horizontal arriba, con las opciones en fila y desplazables. Es la misma página y el mismo componente, no una versión móvil aparte.

3. Los indicadores no tienen puntos de quiebre escritos a mano: `repeat(auto-fit, minmax(190px, 1fr))` los acomoda solos a cuatro, tres, dos o uno según el ancho. Un punto de quiebre por cantidad de columnas habría que mantenerlo cada vez que cambia el ancho de una caja.

4. Las tablas anchas se desplazan dentro de su caja con `overflow-x`. El cuerpo de la página nunca se desplaza de costado.

5. Se puso el meta viewport en el layout raíz. Sin él, el celular achica la página entera y el diseño responsive no se aplica nunca.

6. Se agregó `@media (prefers-reduced-motion: reduce)`, que apaga las transiciones para quien las tenga desactivadas en su sistema.

7. El color nunca es la única señal: el semáforo y la severidad siempre llevan su texto, así que quien no distingue rojo de verde lee el estado igual.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
web/app/globales.css	Paleta, disposición y los puntos de quiebre	c099013
web/app/layout.tsx	El meta viewport	e660c3e

Tabla 7.5. Archivos de la tarea S5.3. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
web/pruebas/componentes.test.tsx	16	Incluye que el semáforo y la severidad nunca muestren sólo color.

Tabla 7.6. Pruebas de la tarea S5.3. Fuente: elaboración propia.

Commits

[e660c3e](#) Semana 5: el panel en Next.js, consumiendo la API real

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

web/app/globales.css líneas 1 a 26 de 508

```

1  /* ShopMetrics - estilos del panel
2    Javier Gomez Avellaneda - UAI 2026
3
4    La paleta y la disposicion salen del prototipo publicado en
5    https://javieravellaneda.github.io/shopmetrics-prototipo/
6
7    La diferencia con el prototipo es que aca todo es responsive desde el
8    arranque, no como arreglo posterior: la semana 9 construye la vista de
9    operaciones para celular sobre estas mismas pantallas. */
10
11  :root {
12    /* --- la paleta del prototipo --- */
13    --violeta: #4F3FC7; /* el color de la marca */
14    --violeta-oscuro: #1E1B4B; /* el fondo del menu */
15    --violeta-medio: #3a3670; /* los separadores dentro del menu */
16    --violeta-claro: #2b2764; /* el hover del menu */
17
18    --verde: #10B981;
19    --ambar: #F59E0B;
20    --rojo: #DC2626;
21
22    --tinta: #222222;
23    --tinta-media: #555555;
24    --tinta-suave: #777777;
25    --linea: #dddddd;
26    --linea-suave: #eeeeee;

```

Figura 7.10. web/app/globales.css. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

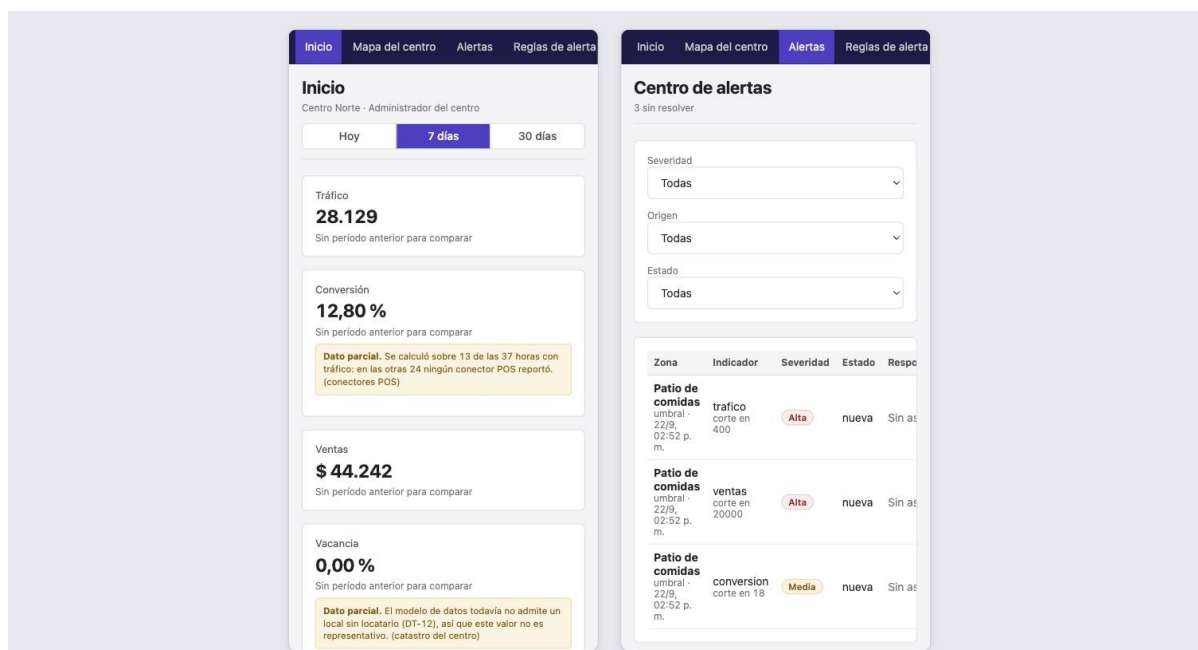


Figura 7.11. Las mismas pantallas a 390 píxeles de ancho: el menú pasa a barra horizontal y los indicadores se apilan. No hay una versión móvil aparte. Fuente: elaboración propia.

Para tener en cuenta

- Por qué ahora y no después: la semana 9 construye la vista de operaciones para celular sobre estas mismas pantallas. Hacerlo después habría significado rehacerlo.
- Verificado redimensionando la ventana del navegador: el menú pasa a barra horizontal, los indicadores se reacomodan y la tabla de integraciones se desplaza dentro de su caja sin empujar la página.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

8.2 S6.1 · Motor de reglas: creación, validación y evaluación periódica

En el plan: «Motor de reglas: creación, validación y evaluación periódica de las condiciones (CU-006).»

Tarjeta del tablero: [#17 — S6.1 · Motor de reglas: creación, validación y evaluación periódica](#)

Qué se hizo

1. Se implementó el alta de la regla con las validaciones del paso 8, cada una fallando por su propio motivo.
2. El período de evaluación y los días de aplicación viajan dentro de condicion como JSON, porque el paso 5 dice que «se almacena dentro de la condición» y la entidad no tiene campos propios para ellos.
3. Un porcentaje mayor que 100 se rechaza: no es un umbral exigente, es uno que no se cumple nunca y deja la regla muda sin que nadie se entere. Un corte de exactamente 100 % sí se acepta: el límite es lo imposible, no lo improbable.
4. Un destinatario sin permiso también se rechaza (CA-2): recibiría un aviso que al abrirlo le daría 403.
5. Se implementó el paso 4: los rangos típicos del indicador, calculados sobre el último mes, para que nadie elija un umbral a ciegas.
6. El curso alternativo CA-1 avisa pero no rechaza: si el corte queda fuera de esos rangos, la regla se crea igual —el usuario puede querer un caso extremo— pero se le dice, porque lo más probable es que se haya equivocado de escala.
7. Se escribió el motor de evaluación, que corre al final de la tarea periódica: evaluar antes de sincronizar usaría los datos de la vuelta anterior.
8. Una regla que se cumple no genera una alerta por vuelta: mientras la anterior siga abierta no se repite. En media hora el responsable tendría cincuenta avisos de lo mismo.
9. La severidad se calcula y no se configura, porque el caso de uso la marca como «calculado por el sistema»: sale de la distancia relativa al corte, así el criterio es el mismo para un indicador que vale 3 y para uno que vale 30.000.
10. «Igual a» tolera el redondeo: un indicador calculado nunca da exactamente el corte, y exigir igualdad estricta haría una regla que no salta jamás.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
api/app/dominios/alertas/reglas.py	Validaciones, rangos típicos y alta	c099013
api/app/dominios/alertas/motor.py	Evaluación, severidad y notificación	c099013
api/app/dominios/alertas/rutas.py	catálogo, rango, reglas, pausar, evaluar	c099013
api/app/tareas.py	La evaluación dentro de la vuelta periódica	c099013

Tabla 8.1. Archivos de la tarea S6.1. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
api/pruebas/test_alertas.py	61	Cubren las ocho validaciones por separado; el CA-1 y el CA-2; el rango típico; que la regla que se cumple genere alerta y la que no, no; las tres comparaciones; que no se repita mientras la anterior siga abierta y que vuelva a disparar cuando se resuelve; la regla pausada; la que no aplica hoy; que sin datos no se dispare nada; la severidad calculada; el aviso a los destinatarios; y que una condición ilegible no tumbe la evaluación.

Tabla 8.2. Pruebas de la tarea S6.1. Fuente: elaboración propia.

Commits

[c099013](#) Semana 6: motor de reglas por umbral y ciclo de vida de la alerta

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A		Turno: Noche	Año: 2026
ShopMetrics					Fecha 22/09/2026

```

api/app/dominios/alertas/motor.py                                líneas 1 a 26 de 178
1  """La evaluacion de las reglas y el nacimiento de la alerta.
2
3  CU-006-001 paso 8: el sistema evalua la regla contra los datos en tiempo real.
4  CU-007-001 paso 2: el sistema genera la alerta y la encola para distribucion.
5
6  Corre dentro de la tarea periodica, despues de sincronizar: evaluar antes
7  usaria datos de la vuelta anterior.
8  """
9  from __future__ import annotations
10
11  import uuid
12  from datetime import datetime, timedelta, timezone
13  from typing import List, Optional
14
15  from sqlalchemy import select
16  from sqlalchemy.orm import Session
17
18  from app.dominios.alertas import reglas as motor_reglas
19  from app.dominios.metricas import servicio as metricas
20  from app.modelos import Alerta, Notificacion, ReglaAlerta, Zona
21
22  UMBRAL = "umbral"
23  NUEVA, EN_REVISION, RESUELTA = "nueva", "en revisión", "resuelta"
24  BAJA, MEDIA, ALTA = "baja", "media", "alta"
25
26  # Los estados en los que la alerta todavia da trabajo. Se usa para no generar

```

Figura 8.3. api/app/dominios/alertas/motor.py. Fuente: elaboración propia.

Para tener en cuenta

- Verificación contra la base real: con una regla de «tráfico mayor que 500 por hora», la primera evaluación generó una alerta —medido 6.368, severidad alta, un destinatario avisado— y las tres siguientes, ninguna.

8.3 S6.2 · Ciclo de vida de la alerta: generación, asignación, resolución y cierre

En el plan: «Ciclo de vida de la alerta: generación, asignación, resolución y cierre.»

Tarjeta del tablero: [#18 — S6.2 · Ciclo de vida de la alerta: generación, asignación, resolución y cierre](#)

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Qué se hizo

1. Primero hubo que destrabar el esquema. El diccionario definía regla_id, locatario_id y asignado_a como obligatorios, aunque las descripciones decían lo contrario —«si aplica», «cuando la alerta es de negocio»—: ninguna usaba la palabra que el generador reconoce. Con eso puesto no se podía guardar ninguna alerta, porque una recién generada no tiene responsable y una del modelo no tiene regla.
2. Se corrigió el diccionario en el Word sin cambiar lo que significaba, se regeneró 001_esquema_inicial.sql y se agregó 004_alerta_opcionales.sql para las bases existentes.
3. Esa migración suma además la restricción que el diccionario describe pero no podía expresar: regla_id y modelo_id son excluyentes. Está en la base y no sólo en el código porque una alerta huérfana no tendría forma de explicar por qué se generó.
4. Se implementaron los tres estados del diccionario: nueva, en revisión, resuelta.
5. Se implementó la asignación del paso 7 del CU-026-001. Tener responsable ya es haber empezado a trabajarla, así que pasa a «en revisión».
6. Se implementaron las tres formas de cerrar, que dejan la misma traza: un falso positivo es información tan valiosa como un problema real, es lo que la semana 7 va a usar para reentrenar.
7. Escalar no cierra: pasa la alerta a otro y le sube la severidad a alta, porque si hizo falta escalarla es más grave de lo que se pensó. Es el CA-2 del CU-007-001.
8. Se implementó el checklist guiado del paso 6, con sus cuatro ítems y sus tres resultados posibles.
9. Una alerta no vuelve de resuelta: reabrirla borraría el tiempo de respuesta que ya se midió. Si el problema sigue, la regla la vuelve a disparar y nace otra. Intentarlo da 409 explicando qué pasó, que es el CA-2 del CU-026-001.
10. El SLA devuelve negativo cuando ya se pasó, y no cero: «te pasaste por cuarenta minutos» es información distinta de «te queda cero». Es el CA-3.
11. Se corrigió un error que encontró la prueba en el navegador: al asignar se ponía sólo la clave foránea, así que alerta.responsable seguía en None y la respuesta devolvía un responsable vacío justo después de asignarlo.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
api/app/dominios/alertas/ciclo.py	Listar, asignar, cerrar, historia y SLA	c099013
base/migraciones/004_alerta_opcionales.sql	Las tres columnas y la restricción	c099013
api/app/modelos.py	Alerta, ReglaAlerta, ResolucionAlerta y Notificacion	c099013
api/app/dominios/alertas/rutas.py	bandeja, detalle, asignar y cerrar	c099013

Tabla 8.3. Archivos de la tarea S6.2. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
api/pruebas/test_alertas.py	61	Que una alerta nueva no tenga responsable; que asignar la pase a «en revisión» y refresque la relación; que avise al nuevo responsable; que no se pueda asignar una resuelta; las tres formas de cerrar; que escalar no cierre y suba la severidad; los límites de 500 caracteres; los ítems y resultados del checklist; que no se pueda cerrar dos veces; que escalar y después resolver deje los dos pasos; y el tiempo de respuesta.
api/pruebas/test_postgres.py	11	Tres contra PostgreSQL que SQLite no puede hacer: que las tres claves admitan nulos, que una alerta sin regla ni modelo se rechace, y que una con las dos también.

Tabla 8.4. Pruebas de la tarea S6.2. Fuente: elaboración propia.

Commits

[c099013](#) Semana 6: motor de reglas por umbral y ciclo de vida de la alerta

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

```

api/app/dominios/alertas/ciclo.py                                líneas 1 a 26 de 254
1  """El recorrido de una alerta: nace, se asigna, se trabaja y se cierra.
2
3  CU-007-001 Atender una alerta
4  CU-026-001 Centro de alertas criticas
5
6  Los tres estados son los del diccionario de datos: nueva, en revision y
7  resuelta. Los dos casos de uso los nombran distinto —uno dice abierta/en
8  proceso/resuelta y el otro activa/en seguimiento/resuelta— y esa diferencia
9  esta anotada como DT-14.
10
11  Una alerta no vuelve de resuelta. Reabrir la borraría el tiempo de respuesta que
12  ya se midió; si el problema sigue, la regla la vuelve a disparar y nace otra.
13  """
14  from __future__ import annotations
15
16  import uuid
17  from datetime import datetime, timezone
18  from typing import List, Optional, Tuple
19
20  from sqlalchemy import select
21  from sqlalchemy.orm import Session
22
23  from app.dominios.alertas.motor import (ABIERTAS, ALTA, EN_REVISION, MEDIA,
24                                          NUEVA, RESUELTA)
25  from app.modelos import (Alerta, Notificacion, ResolucionAlerta, Usuario, Zona)

```

Figura 8.4. api/app/dominios/alertas/ciclo.py. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				Fecha 22/09/2026
	Materia: Trabajo Final De Ingeniería				
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

```

base/migraciones/004_alerta_opcionales.sql                                líneas 1 a 26 de 32
1  -- Tres claves foraneas de `alerta` que tenian que admitir nulos.
2  --
3  -- El diccionario de datos del punto 10.5.8 ya lo decia en prosa --"si aplica",
4  -- "cuando la alerta es de negocio"-- pero no con la palabra que el generador
5  -- del esquema reconoce, asi que las columnas salieron NOT NULL. Con eso puesto
6  -- no se puede guardar ninguna alerta:
7  --
8  -- * `asignado_a`: una alerta recién generada no tiene responsable. El paso 7
9  --   del CU-026-001 es justamente donde el gerente lo asigna.
10 -- * `regla_id`: una alerta del modelo no nace de ninguna regla de umbral. El
11 --   paso 2 del CU-007-001 la genera sin regla.
12 -- * `locatario_id`: solo se completa cuando la alerta es de negocio.
13 --
14 -- El diccionario quedo corregido y `001_esquema_inicial.sql` se regenero, asi
15 -- que una base creada desde cero ya sale bien. Esta migracion es para las que
16 -- ya existen.
17 --
18 ALTER TABLE alerta ALTER COLUMN regla_id      DROP NOT NULL;
19 ALTER TABLE alerta ALTER COLUMN locatario_id  DROP NOT NULL;
20 ALTER TABLE alerta ALTER COLUMN asignado_a    DROP NOT NULL;
21 --
22 -- Una alerta nace de una regla de umbral o del modelo, nunca de las dos ni de
23 -- ninguna. El diccionario lo dice en las descripciones de los dos campos; sin
24 -- la restriccion, nada impide guardar una alerta huérfana.
25 ALTER TABLE alerta DROP CONSTRAINT IF EXISTS ck_alerta_origen;
26 ALTER TABLE alerta ADD CONSTRAINT ck_alerta_origen CHECK (

```

Figura 8.5. base/migraciones/004_alerta_opcionales.sql. Fuente: elaboración propia.

Para tener en cuenta

- El cambio de multiplicidades en el diagrama de clases quedó anotado como hallazgo 10 en modelo-ea/CORRECCIONES-PENDIENTES.md.
- El estado se llama distinto en tres lugares: el diccionario dice nueva/en revisión/resuelta, el CU-007-001 dice abierta/en proceso/resuelta y el CU-026-001 dice activa/en seguimiento/resuelta. Se usaron los del diccionario y quedó como DT-14.

Deuda técnica detectada

- [Tarjeta #57 del tablero](#)
- [Tarjeta #58 del tablero](#)

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			Fecha 22/09/2026
	Sede: Centro	Comisión: 5° A	Turno: Noche	
ShopMetrics				

8.4 S6.3 · Centro de alertas en el panel, con filtros por origen y severidad

En el plan: «Centro de alertas en el panel, con filtros por origen y severidad (CU-026).»

Tarjeta del tablero: [#19 — S6.3 · Centro de alertas en el panel, con filtros por origen y severidad](#)

Qué se hizo

1. Se armaron dos pantallas: el centro de alertas y la configuración de reglas.
2. La lista ordena por severidad y, dentro de cada una, primero las más viejas: la que lleva más esperando es la que está más cerca de pasarse del objetivo.
3. Se implementaron los tres filtros del paso 4: severidad, origen y estado. Los vacíos no viajan en la consulta.
4. La ficha va al costado de la lista, no en otra página. El gerente está mirando varias alertas a la vez; sacarlo de la bandeja para atender una y devolverlo después le hace perder el contexto que vino a buscar.
5. El tiempo se escribe como se lee —«1 h 20 min», no «80»— y conserva el signo: una alerta vencida dice «vencido por 40 min» en rojo, con una franja al costado de la fila que se ve de rojo al recorrer la lista.
6. Curso alternativo CA-1: sin alertas abiertas no se muestra una tabla vacía sino «Todo en orden» con la marca del último chequeo. Una lista vacía sin explicación deja al gerente sin saber si no hay nada o si la consulta falló.
7. La pantalla de reglas trae el paso 4 al lado del campo: antes de elegir el corte, el usuario ve qué viene midiendo ese indicador. Y el rótulo del campo trae la unidad, así que dice «(porcentaje)» o «(personas)» según el indicador elegido.
8. Las opciones salen del catálogo de la API, no de una lista propia: si el backend suma un canal, aparece en el panel sin tocar nada.

Archivos que quedaron escritos

Archivo	Qué resuelve	Commit
web/app/panel/alertas/page.tsx	La bandeja con sus filtros	c099013
web/componentes/FichaDeAlerta.tsx	Detalle, historia, asignar y cerrar	c099013

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

web/componentes/Severidad.tsx	La cinta de severidad y los minutos	c099013
web/app/panel/reglas/page.tsx	El formulario del CU-006-001	c099013
web/app/globales.css	Los estilos de severidad y de la ficha	c099013

Tabla 8.5. Archivos de la tarea S6.3. Cada nombre enlaza al archivo en el commit indicado. Fuente: elaboración propia.

Pruebas

Archivo de pruebas	Cuántas	Qué cubren
web/pruebas/alertas.test.tsx	23	Los minutos como se leen; que la severidad nunca sea sólo color; que cada alerta salga con su zona, indicador y responsable; que se cuenten las abiertas; que se marquen las atrasadas; el cartel de «Todo en orden»; que los filtros viajen y que sacarlos los quite; que las opciones salgan del catálogo; los rangos típicos; el aviso de CA-1 y el rechazo de la API.

Tabla 8.6. Pruebas de la tarea S6.3. Fuente: elaboración propia.

Commits

[c099013](#) Semana 6: motor de reglas por umbral y ciclo de vida de la alerta

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				Fecha 22/09/2026
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

web/app/panel/alertas/page.tsx líneas 1 a 26 de 160

```

1  /* CU-026-001 Centro de alertas criticas.
2  *
3  * La bandeja ordenada por severidad, con los filtros del paso 4 y la ficha de
4  * la alerta abierta al costado: reasignar, escalar o resolver sin salir de la
5  * lista, porque el gerente esta mirando varias a la vez.
6  */
7  "use client";
8
9  import { useState } from "react";
10 import Severidad, { enMinutos } from "@componentes/Severidad";
11 import FichaDeAlerta from "@componentes/FichaDeAlerta";
12 import { usarPedido } from "@componentes/usarPedido";
13 import { bandeja, type FiltrosDeAlertas } from "@servicios/api";
14 import type { AlertaVista } from "@servicios/tipos";
15
16 const SEVERIDADES = ["alta", "media", "baja"];
17 const ORIGENES = ["umbral", "ML"];
18 const ESTADOS = ["nueva", "en revisión", "resuelta"];
19
20 function cuando(marca: string): string {
21   return new Date(marca).toLocaleString("es-AR", {
22     day: "2-digit", month: "2-digit", hour: "2-digit", minute: "2-digit",
23   });
24 }
25
26 export default function CentroDeAlertas() {

```

Figura 8.6. web/app/panel/alertas/page.tsx. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				Fecha 22/09/2026
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

```

web/componentes/FichaDeAlerta.tsx                                líneas 1 a 26 de 243
1  /* La ficha de una alerta: su detalle, su historia y que hacer con ella.
2  *
3  * CU-026-001 pasos 6 y 7: el detalle, el SLA y la reasignacion.
4  * CU-007-001 pasos 6 a 8: el checklist guiado y el cierre.
5  */
6  "use client";
7
8  import { useState } from "react";
9  import Severidad, { enMinutos } from "../Severidad";
10 import { usarPedido } from "../usarPedido";
11 import { alerta as traerAlerta, asignar, cerrar, usuarios } from "@servicios/api";
12
13 type Props = { id: string; alCerrarFicha: () => void; alCambiar: () => void };
14
15 const FORMAS = [
16   { valor: "resolver", texto: "Resolver" },
17   { valor: "falso positivo", texto: "Falso positivo" },
18   { valor: "escalar", texto: "Escalar" },
19 ];
20
21 const RESULTADOS = ["verificado", "problema encontrado", "no aplica"];
22 const LARGO_MAXIMO = 500;
23
24 function cuando(marca: string): string {
25   return new Date(marca).toLocaleString("es-AR", {
26     day: "2-digit", month: "2-digit", hour: "2-digit", minute: "2-digit",

```

Figura 8.7. web/componentes/FichaDeAlerta.tsx. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingenieria				Fecha 22/09/2026
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

ShopMetrics

Inicio

Mapa del centro

MÓDULOS

Alertas

Reglas de alerta

Integraciones

Administrador

Administrador del centro

Cerrar sesión

Centro de alertas

3 sin resolver

Severidad

Todas

Origen

Todas

Estado

Todas

Zona	Indicador	Severidad	Estado	Responsable	Antigüedad	Objetivo
Patio de comidas umbral - 22/9, 02:52 p. m.	trafico corte en 400	Alta	nueva	Sin asignar	1 min	29 min
Patio de comidas umbral - 22/9, 02:52 p. m.	ventas corte en 20000	Alta	nueva	Sin asignar	1 min	29 min
Patio de comidas umbral - 22/9, 02:52 p. m.	conversion corte en 18	Media	nueva	Sin asignar	1 min	1 h 59 min

Figura 8.8. Centro de alertas, ordenado por severidad y con los filtros del paso 4. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Trabajo Final De Ingeniería				Fecha 22/09/2026
	Carrera: Ingeniería en Sistemas				
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026	
ShopMetrics					

ShopMetrics

Inicio

Mapa del centro

MÓDULOS

Alertas

Reglas de alerta

Integraciones

Centro de alertas

3 sin resolver

Severidad

Todas

Origen

Todas

Estado

Todas

Zona	Indicador	Severidad	Estado	Responsable	Antigüedad	Objetivo	
Patio de comidas	umbral - 22/9, 02:52 p. m.	tráfico	Alta	nueva	Sin asignar	1 min	29 min
Patio de comidas	umbral - 22/9, 02:52 p. m.	ventas	Alta	nueva	Sin asignar	1 min	29 min
Patio de comidas	umbral - 22/9, 02:52 p. m.	conversion	Media	nueva	Sin asignar	1 min	1 h 59 min

Alerta

Cerrar

Zona

Patio de comidas

Indicador

tráfico

Severidad

Alta

Estado

nueva

Generada

22/9, 02:52 p. m.

Responsable

Sin asignar

Objetivo

quedan 29 min

Asignar

Elegir responsable...

Cerrar

Resolver

Falso positivo

Escalar

Checklist

verificar escaleras mecánicas

Sin revisar

verificar climatización

Sin revisar

verificar pasillos bloqueados

Figura 8.9. La ficha al costado de la lista: detalle, SLA, asignación y checklist guiado. Fuente: elaboración propia.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática					
	Materia: Trabajo Final De Ingeniería					Fecha 22/09/2026
	Carrera: Ingeniería en Sistemas					
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026		
ShopMetrics						

ShopMetrics
Inicio
Mapa del centro
MÓDULOS
Alertas
Reglas de alerta
Integraciones

Administrador
Administrador del centro
Cerrar sesión

Reglas de alerta

Cuando el indicador cruza el umbral, nace una alerta

Nueva regla

Indicador: conversion
Condición: mayor que
Valor de corte (porcentaje):
Evaluar: por hora

En el último mes estubo entre 7,84 y 15,60, con un promedio de 12,83.

Días de aplicación:
☒ lunes
☒ martes
☒ miércoles
☒ jueves
☒ viernes
☒ sábado
☒ domingo
Avisarle a: Elegir...
Por: push móvil

Crear la regla

Reglas definidas

Indicador	Condición	Evalúa	Días	Avisa a	Estado	
trafico	mayor que 400 personas	por hora	todos	Administrador	activa	Pausar
conversion	menor que 18 porcentaje	diario	todos	Administrador	activa	Pausar
ventas	mayor que 20000 pesos	semanal	todos	Administrador	activa	Pausar

Figura 8.10. Configuración de reglas. Debajo del indicador se muestran sus rangos típicos del último mes. Fuente: elaboración propia.

Para tener en cuenta

- Verificación en el navegador: la bandeja mostró las tres alertas; asignando desde la ficha, la fila y la ficha pasaron a «en revisión»; cerrando con el checklist, la alerta pasó a resuelta con «Se resolvió en 2 min» y la traza completa; creando una regla de conversión con corte en 95 —el histórico llega a 37,93— apareció el aviso de CA-1.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				

Fecha
22/09/2026

9. Deuda técnica detectada durante la construcción

Construir el sistema contra la especificación dejó al descubierto quince diferencias entre el documento de análisis, los diagramas y lo que el sistema necesita para funcionar. Ninguna se resolvió por las malas: cada una quedó registrada con qué dice cada fuente, por qué es un problema y qué hay que hacer.

El detalle completo está en el archivo modelo-ea/CORRECCIONES-PENDIENTES.md del repositorio de documentación, y cada hallazgo tiene además su tarjeta en el tablero. Se separan en dos grupos, porque se corrigen en lugares distintos.

9.1 Correcciones sobre el modelo de Enterprise Architect

Son diferencias entre los diagramas y el modelo de datos. Se corrigen sobre el archivo shopMetrics.eapx, que sólo abre en la máquina con Windows, y después hay que volver a exportar los diagramas.

N.º	Hallazgo
1	El doble factor no tiene dónde guardarse en el modelo de datos.
2	Rol figura como enumeración en un diagrama y como tabla en el modelo.
3	La asociación con los centros pierde su clase asociativa.
4	La enumeración de operaciones dice SIN_ACCESO donde el caso de uso dice eliminación.
5	Falta el campo descripcion en la entidad rol del diccionario.
6	El atributo del permiso se llama categoría en el diagrama y recurso en el modelo.
7	Verificar que la notificación de cambio de permisos esté en el diagrama de secuencia.
10	Las multiplicidades hacia Alerta deben pasar de 1 a 0..1 en tres asociaciones.

Tabla 9.1. Correcciones pendientes sobre el modelo de Enterprise Architect. Fuente: elaboración propia.

9.2 Huecos del documento de análisis

Éstos no son diferencias entre el diagrama y el modelo: son cosas que el documento no dice o que dice de dos maneras distintas. La corrección va sobre el informe, no sobre los diagramas.

N.º	Hallazgo	Cómo se resolvió mientras tanto
8	Un local vacante no se puede representar, y la vacancia es uno de los cuatro indicadores del tablero.	El indicador se calcula y viaja marcado como dato parcial.
9	El paso 5 del CU-001-001 pide una latencia promedio y a la vez dice que no se almacena.	Se muestra la latencia del sondeo del momento.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

11	El estado de la alerta se llama distinto en el diccionario, en el CU-007-001 y en el CU-026-001.	El código usa los valores del diccionario.
12	alerta.zona_id es obligatorio, pero una regla sobre un indicador del centro no es de ninguna zona.	La alerta se cuelga de la primera zona del centro.

Tabla 9.2. Huecos del documento de análisis detectados al construir. Fuente: elaboración propia.

9.3 Una corrección que hubo que hacer sobre la marcha

El hallazgo 10 es el único que no se pudo dejar anotado para después, porque bloqueaba la construcción de la semana 6. El diccionario definía tres claves foráneas de la entidad alerta como obligatorias, aunque sus propias descripciones decían lo contrario: «si aplica» para la regla que la originó, «cuando la alerta es de negocio» para el locatario. Ninguna de las dos usaba la palabra que el generador del esquema reconoce para emitir una columna que admita nulos.

Con las tres columnas obligatorias no se podía guardar ninguna alerta: una recién generada no tiene responsable asignado —el paso 7 del CU-026-001 es justamente donde se lo asigna— y una generada por el modelo no nace de ninguna regla de umbral. Se corrigieron las descripciones del diccionario sin cambiar lo que significaban, se regeneró el esquema y se agregó una migración para las bases ya creadas.

	UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
	Materia: Trabajo Final De Ingeniería			
	Carrera: Ingeniería en Sistemas			
	Sede: Centro	Comisión: 5° A	Turno: Noche	Año: 2026
ShopMetrics				
Fecha 22/09/2026				

10. Estado al cierre de la semana 6

	Estado
Tareas del plan cerradas	19 de 19, con evidencia en cada tarjeta
Casos de uso con endpoints	10 de los 31 especificados
Pruebas de la API	191, todas en verde
Pruebas del panel	76, todas en verde
Nivelación con el documento	En verde
Entidades en la base	30 tablas, 43 claves foráneas
Hipertablas de TimescaleDB	2, con 2 agregados continuos
Migraciones aplicadas	4, reproducibles desde cero
Deuda técnica registrada	15 hallazgos, todos con tarjeta

Tabla 10.1. Estado del sistema al cierre de la semana 6. Fuente: elaboración propia.

10.1 Lo que sigue

Las semanas 7 a 11 del plan continúan con la detección de anomalías y el score de vacancia, el portal del locatario, la vista de operaciones para celular, la auditoría y el despliegue, y la especificación de los casos de prueba. Las tarjetas ya están creadas en el tablero con el mismo nivel de detalle que las de estas seis semanas.

10.2 Repositorios

[shopmetrics](#) el sistema, el panel, los simuladores y el generador

[ShopMetrics-TFI](#) la documentación, los diagramas y este informe